SkillLLM_v2: Provenance-Aware Skill Extraction, Normalization, and Contextual Career Intelligence

Technical Research Report

Prepared for CSC 895 / CSC 898 and Professor Chen

San Francisco State University

Department of Computer Science

September 2026

Administrative note. This research report is thesis-quality but is not a signed submission copy. The author name, final course designation, committee information, and approval-page format must be confirmed with the department before formal submission.

Abstract

Skill extraction is a prerequisite for systems that compare a person's evidence-backed capabilities with job requirements and learning opportunities. Yet a useful extraction system must do more than emit plausible labels: it must preserve the source span, distinguish hard and soft skills, remain robust when auxiliary components disagree, and expose enough provenance for later matching and recommendation. This report presents SkillLLM_v2, a modular research system for span-level skill extraction, canonicalization, profile construction, graph-structured explanation, job matching, and course recommendation.

The central extraction component is a Mistral Small 3.1 24B Instruct model adapted with a low-rank adapter on the local SkillSpan export. The audited 80-epoch, two-GPU run obtains exact-span F1 of 0.670069, token F1 of 0.787886, hard-skill F1 of 0.726450, and soft-skill F1 of 0.604046 on its persisted full-test artifact. In the same evaluation record, the BERT and zero-shot GLiNER baselines achieve exact-span F1 of 0.059150 and 0.075833 respectively. These results make the adapted Mistral model the extraction anchor rather than evidence that adding more modules automatically improves performance.

SkillLLM_v2 therefore uses provenance-aware, non-regressive composition. Candidate merging protects anchor spans, taxonomy linking is attach-only, and a gate selects wrapper predictions only when they stay within configured exact-span and soft-skill tolerances. The full wrapper is accepted as non-regressive and records exact-span F1 of 0.671102, but the difference is small and does not establish a general improvement claim. The implementation also constructs evidence-preserving profile summaries, deterministic cached context packets, and an in-memory skill graph. These downstream paths are implemented and smoke-tested; however, labelled job-ranking and course-ranking evaluations are unavailable, so no effectiveness claim is made for them.

The report contributes a detailed code-to-artifact audit, a transparent experimental account, reproducible system diagrams, and a research finding of practical importance: a modular career-intelligence pipeline should treat high-quality extraction as an invariant to protect, while using normalization, context, and graph components to enrich provenance and

explanation rather than silently rewriting reliable spans.

Acknowledgements

This report was prepared for the SkillLLM_v2 research update. The author thanks Professor Chen for guidance on the scope of the culminating research deliverable and the contributors of the SkillSpan benchmark for making span-level hard- and soft-skill evaluation possible.

Contents

Abstract **i**

Acknowledgements **iii**

Abbreviations **xi**

1 Introduction **1**

1.1 Motivation 1

1.2 Research problem 2

1.3 Research questions 3

1.4 Contributions 4

1.5 Thesis organization 4

2 Background and Related Work **5**

2.1 Skill extraction and skill inventories 5

2.2 Named-entity recognition and span modelling 6

2.3 Domain adaptation and parameter-efficient fine-tuning 6

2.4 Taxonomy linking and retrieval 7

2.5 Graphs, matching, and contextual career intelligence 7

2.6 Additional methodological perspectives 8

2.7 Research gap and positioning 9

3 Problem Formulation **10**

3.1 Documents, spans, and labels 10

3.2 Metrics 11

3.3 Anchor-preserving composition 11
3.4 Canonical profile and downstream representations 12
3.5 Deterministic matching and recommendation 12

4 SkillLLM_v2 System Architecture 14
4.1 Architecture principle and implementation status 14
4.2 Input contract and record schema 15
4.3 Mistral LoRA extraction anchor 16
4.4 Auxiliary extraction, merging, and refinement 17
4.5 Safe-wrapper selection 17
4.6 Canonicalization and taxonomy attachment 18
4.7 Context, graph, and downstream services 19
4.8 Architecture summary 20

5 Dataset and Experimental Methodology 21
5.1 Evidence-first methodology 21
5.2 Dataset and local representation 21
5.3 SFT data preparation and prompt contract 22
5.4 Training procedure 23
5.5 Baselines and comparison discipline 24
5.6 Extraction evaluation 25
5.7 Normalization and downstream methodology 25
5.8 Reproducibility procedure 26

6 Experiments 27
6.1 Experimental objectives 27
6.2 Experiment E1: extraction baselines 27
6.3 Experiment E2: historical wrapper diagnosis 28
6.4 Experiment E3: safe-wrapper variants 28
6.5 Experiment E4: taxonomy attachment 29
6.6 Experiment E5: profile, graph, and recommendation vertical slice 30

6.7 Artifact tracking and protocol fidelity 30

7 Results **32**

7.1 Primary extraction results 32

7.2 Wrapper results and selection semantics 33

7.3 Linking and downstream results 34

7.4 Answer to the research questions 34

8 Error Analysis and Non-Regressive Design **36**

8.1 Why an error analysis is necessary 36

8.2 Historical destructive post-processing 36

8.3 From diagnosis to control points 37

8.4 Current safe-wrapper interpretation 38

8.5 Likely remaining error categories 38

8.6 Error-analysis protocol for future runs 39

9 Context Intelligence: Profiles, Graphs, Matching, and Recommendations **40**

9.1 From a span list to a durable profile 40

9.2 Context compaction policy 41

9.3 Why evidence preservation matters for compaction 42

9.4 Knowledge graph representation 42

9.5 Job matching and skill-gap detection 43

9.6 Course recommendation 44

9.7 A staged deployment interpretation 45

10 Limitations, Threats to Validity, and Future Work **46**

10.1 Claim boundary 46

10.2 Dataset and evaluation limitations 46

10.3 Model and training limitations 47

10.4 Normalization and ontology limitations 47

10.5 Context, graph, and recommendation limitations 48

10.6 Reproducibility and report-format limitations 48

10.7 Future work 49

11 Conclusion **50**

A Reproducibility Record **51**

A.1 Environment evidence 51

A.2 Data and source checkpoints 52

A.3 Representative commands 52

A.4 Artifact verification sequence 53

A.5 Tests and their interpretation 53

B Prompt and Data-Contract Details **54**

B.1 Extraction prompt design 54

B.2 Skill candidate contract 54

B.3 Profile and context contract 55

B.4 Matching and recommendation output contract 55

C Audit Trail and Artifact Index **57**

C.1 Primary source of truth 57

C.2 Known metric conflict 58

C.3 Report maintenance checklist 58

List of Figures

1.1 Overall SkillLLM_v2 structure. The architecture distinguishes implemented local modules from unevaluated claims about recommendation effectiveness. 2

4.1 Implemented extraction path. The anchor creates source-grounded candidates; GLiNER and dictionary components can add candidates but do not replace the anchor by default. 16

4.2 Safe-wrapper design. Protection occurs twice: candidate merging preserves Mistral-originated spans, and the evaluation gate can fall back to cached anchor predictions. 18

4.3 Attach-only normalization. The canonical fields enrich a candidate but do not rewrite the extraction fields used by span evaluation. 19

4.4 Implemented in-memory knowledge-graph schema. A missing skill is derived from a user/job comparison; it is not an asserted source fact. 20

5.1 Training and evaluation pipeline traced to the local data-preparation and trainer modules. The persisted evaluation artifact is distinct from smoke-mode fallback. 24

5.2 Evaluation path for extraction and wrapper selection. Downstream ranking metrics use a separate, currently unavailable labelled protocol. 25

6.1 Artifact and provenance pipeline. The timestamp check is a practical safeguard, not an immutable experiment-management system. 31

7.1 Exact-span and token F1 from primary persisted artifacts. The full wrapper's exact-span difference from Raw Mistral is 0.001033 and should not be interpreted as a robust gain. 33

9.1 Implemented evidence-preserving profile summarization. It aggregates explicit candidates and records conflicts instead of inferring or silently resolving them. 41

9.2 Implemented context-compaction policy. Stable profile structures are cached; dynamic job information and graph explanations remain separate responsibilities. 42

9.3 Skill graph relation types used by the current vertical slice. Direct edges are constructed from supplied records; a missing skill is derived. 43

9.4 Implemented deterministic matching and gap-detection flow. Optional semantic, experience, and preference fit fields are caller-supplied rather than learned by the repository. 44

9.5 Implemented course-recommendation flow and its explicit evaluation boundary. P@5 is unavailable because no labelled course relevance data are persisted. 45

10.1 Evidence status across SkillLLM_v2. A complete vertical slice is not the same as a complete outcome evaluation. 46

List of Tables

4.1 Evidence status by subsystem. "Implemented" describes local source and artifacts, not a blanket production-readiness claim. 15

5.1 Local SkillSpan export used by the Stage-2 pipeline. The local JSON line counts match the dataset README. 22

5.2 Verified Mistral LoRA settings from `skillllm_v2_training_80ep_2gpu.yaml` and the persisted training summary. 23

6.1 Safe-wrapper gate results. Acceptance means that the candidate clears configured non-regression floors; it does not mean that it reached the historical improvement target. 29

7.1 Primary full-test extraction artifacts. Values are copied from persisted `metrics.json` files, not inferred from prose reports. 32

8.1 Historical error diagnosis from `wrapper_error_analysis.md`. These values describe a prior wrapper analysis and are not pooled with the newer primary table. 37

C.1 Claim-to-artifact index. 57

Abbreviations

Term	Meaning
CAG	Cache-augmented context in the SkillLLM_v2 implementation
DDP	Distributed data parallel training
F1	Harmonic mean of precision and recall
GLiNER	Generalist model for open-label named-entity recognition
LoRA	Low-rank adaptation
NER	Named-entity recognition
PEFT	Parameter-efficient fine-tuning
RAG	Retrieval-augmented generation
SFT	Supervised fine-tuning

Chapter 1

Introduction

1.1 Motivation

Job advertisements, resumes, course descriptions, and career histories all describe capabilities in natural language. A person may write "built data pipelines in Python and SQL," a vacancy may request "cloud deployment and stakeholder communication," and a course description may state that it teaches distributed systems. A career-support system cannot reliably compare these texts by string overlap alone. The terms may be abbreviated, may refer to the same canonical capability through aliases, and may encode different evidence strengths. At the same time, a system that invents a capability, loses the text span that justified it, or modifies a correct prediction while attempting to normalize it creates an error that propagates to matching and recommendation.

Skill extraction is therefore a useful intermediate representation for computational labour-market analysis. SkillSpan formalizes the task as span-level identification of hard and soft skills in English job postings and supplies expert annotation guidelines rather than a fixed inventory-only label space [48]. Recent surveys position this task at the intersection of information extraction, taxonomy alignment, and labour-market analysis [41]. The local SkillSpan export is particularly suitable for a research system because it retains tokens, two skill-related BIO tag sequences, document delimiters, and a source field. It permits a span-focused comparison in which surface form, boundaries, and label must all be handled explicitly.

Large language models make structured extraction attractive because one model can produce a variable number of typed spans in a JSON schema. However, a generative output is not automatically an auditable record. It can include paraphrases instead of substrings, fail to parse, produce ambiguous labels, or make a broadly reasonable but unsupported

inference. Classical and neural sequence labellers have complementary properties: they are often tied to a fixed labelling interface and can be evaluated directly at token or span level, but they do not by themselves solve canonicalization, evidence preservation, or downstream context construction. The design challenge is thus not simply to add components around an LLM. It is to make every component safe with respect to the strongest available extraction evidence.

This report documents SkillLLM_v2, a codebase that implements a Mistral LoRA extraction path, auxiliary GLiNER and dictionary candidates, provenance-aware merging, rule-based verification and calibration interfaces, attach-only taxonomy linking, structured profile compaction, in-memory graph construction, deterministic matching, course recommendation, and artifact gates. The report treats these as distinct claims. Some are evaluated with persisted full-test extraction metrics; some are exercised by unit tests and demo artifacts; and some remain future evaluation work. This separation matters in an advisor-facing report because a runnable architectural path is valuable, but it is not equivalent to a validated product outcome.

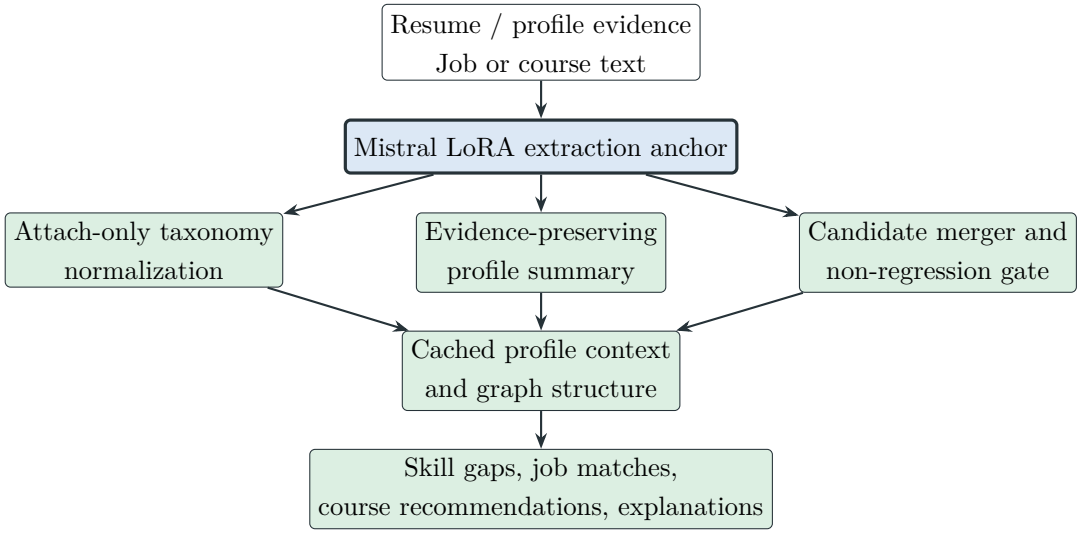


Figure 1.1: Overall SkillLLM_v2 structure. The architecture distinguishes implemented local modules from unevaluated claims about recommendation effectiveness.

1.2 Research problem

The main problem addressed in this work is: *How can a domain-adapted language model be combined with provenance-aware normalization and contextual structure without allowing*

downstream processing to silently degrade reliable span extraction? This formulation intentionally places reliability before architectural breadth. It does not assume that a graph, retriever, verifier, or taxonomy linker improves F1 merely because it is present. Instead, it asks whether a high-quality anchor can remain the source of truth for extraction while other modules add canonical identifiers, evidence-bearing user memory, gap explanations, and bounded decision support.

The code and artifacts give a concrete reason for this framing. The persisted full-test Mistral LoRA artifact has exact-span F1 of 0.670069 and token F1 of 0.787886. The corresponding BERT and GLiNER artifacts score 0.059150 and 0.075833 exact-span F1. The full CAG/graph wrapper records 0.671102 exact-span F1, a difference of 0.001033 over the anchor. That small difference is inside the configured gate tolerance and is sufficient to support a non-regression decision in this snapshot; it is not sufficient evidence for a general claim that a complex wrapper improves extraction. A separate historical error-analysis report records substantial destructive boundary edits in an earlier path. The system design responds to that observation by preserving anchor candidates and by making fallback explicit.

1.3 Research questions

The report answers the following questions to the extent supported by local evidence.

1. **RQ1:** How effectively does the persisted Mistral LoRA extraction path identify hard and soft skill spans in the local SkillSpan evaluation path?
2. **RQ2:** How do the BERT baseline, zero-shot GLiNER baseline, and wrapper variants compare with the Mistral anchor under the stored metrics?
3. **RQ3:** What implementation rules make auxiliary candidate generation, boundary refinement, verification, and normalization non-destructive with respect to anchor extraction?
4. **RQ4:** What evidence-preserving representation can transform extracted skills into a compact profile and contextual packet suitable for later retrieval and explanation?
5. **RQ5:** Which downstream graph, matching, and recommendation capabilities are implemented, and which still lack labelled outcome evaluation?

These questions deliberately do not ask whether the system achieves better job matching or recommendation quality. The repository contains no relevance labels for such a claim: job nDCG@10 and course precision@5 are both recorded as unavailable. The design and

implementation can be assessed now; ranking effectiveness must be assessed later with a labelled protocol.

1.4 Contributions

The first contribution is an evidence-led account of a two-GPU 80-epoch Mistral LoRA run over the local SkillSpan export. Its configuration, checkpoints, training summary, generated predictions, and metrics are persisted. The second is a provenance-aware candidate policy. Each valid skill record carries a surface form, label, offsets, source, confidence, evidence, canonical fields, and metadata. Mistral-originated candidates are protected; unanchored additions must clear configured confidence and alias-evidence conditions; and the wrapper gate compares candidate metrics with the anchor before choosing output predictions.

The third contribution is an implementation-level design for enriching extraction without conflating enrichment with accuracy. Taxonomy linking attaches information instead of rewriting offsets or labels. Profile summarization aggregates supplied extraction evidence rather than inferring hidden skills. Context compaction distinguishes stable cached information from dynamic job text and graph paths. The graph, matcher, gap detector, and recommender produce inspectable, deterministic records. Finally, the report contributes the audit in `docs/thesis/codebase_audit.md`, system figures, a result table backed by JSON artifacts, and explicit limitations that prevent the thesis from overstating the current system.

1.5 Thesis organization

Chapter 2 situates the project in skill extraction, span modelling, parameter-efficient adaptation, normalization, retrieval, and graph-based career systems. Chapter 3 defines the representations and metrics. Chapter 4 maps the actual source tree to the system architecture. Chapter 5 and Chapter 6 describe data preparation, training, evaluation, and artifact handling. Chapter 7 reports the verified metrics. Chapter 8 discusses wrapper regression and the non-regression response. Chapter 9 documents the profile, context, graph, and matching layers. The final chapters state limitations, future work, and conclusions.

Chapter 2

Background and Related Work

2.1 Skill extraction and skill inventories

Skill extraction identifies capability mentions in unstructured text. It is related to named-entity recognition, but the boundary of a skill can be long, compositional, and context dependent. A phrase such as "experience with scalable distributed systems" may include a tool, a concept, an experience qualifier, or several nested skills depending on the annotation rules. SkillSpan addresses this problem by annotating hard and soft skills at span level across 14.5K English job-posting sentences with more than 12.5K annotated spans [48]. It is particularly relevant here because the local export maintains the hard/soft distinction rather than collapsing all skills into one universal class.

Earlier systems connected extraction with normalization. The SKILL system treated identification and normalization as a linked problem at scale [52]; later work considers large-scale occupational normalization and extreme multi-label retrieval from job descriptions [17, 3]. This distinction is still important. Extraction answers whether a source says "AWS," whereas normalization decides whether a surface form maps to a canonical concept such as Amazon Web Services. Conflating the two can hide an extraction error behind a plausible identifier. SkillLLM_v2 adopts an explicit separation: its linker is attach-only, so canonicalization should not affect span-level extraction F1.

The Skill-LLM lineage demonstrates that general-purpose LLMs can be adapted for skill extraction [16]. Related job-market research explores LLM extraction, synthetic job-posting generation, and nearest-neighbour occupational skill extraction [37, 36, 49]. These efforts motivate a generative JSON interface but do not remove the need for controlled local evaluation. In this report, the substantive model claim is limited to the stored Stage-2 Mistral results on the local data path, not a direct benchmark claim against papers using different

pre-processing, model versions, or splits.

2.2 Named-entity recognition and span modelling

Sequence tagging is a useful baseline for extraction. BiLSTM-CRF and character-aware variants established effective NER architectures before transformer encoders [20, 26, 35]. BERT made contextual token representations broadly available for downstream classification and sequence labelling [9]; RoBERTa and XLM-R extended the pre-training design space [34, 7]. A BERT baseline is consequently an important control in this project, even though its local artifact is explicitly a CPU-fast baseline and does not match the Mistral result.

Span-specific methods emphasise that boundaries, not only token labels, determine correctness. SpanBERT pre-trains representations that focus on contiguous spans [23], and SpanNER formulates recognition through span prediction [10]. DyGIE++ extends contextual span representations to entities, relations, and events [43]. Such work is conceptually aligned with SkillSpan evaluation: a semantically reasonable phrase is still incorrect under exact matching when either boundary or label differs. SkillLLM_v2 therefore reports exact, partial, token, hard-skill, and soft-skill metrics rather than relying on a single aggregate alone.

GLiNER is especially relevant as an auxiliary source because it supports generalist, open-label entity recognition with a bidirectional transformer [47]. In the audited repository, however, the standalone GLiNER result is zero-shot/few-shot; fine-tuning was skipped. The thesis treats it as a recall-oriented candidate source and an honest baseline, not as evidence of a trained competitor. Generative approaches such as GPT-NER show that LLMs can produce entity structures directly [44]. Their principal risk in an extraction pipeline is not merely an invalid JSON string; it is a fluent string that fails to align to the input or rewrites a source span. The alignment and validation code in SkillLLM_v2 address this operational issue.

2.3 Domain adaptation and parameter-efficient fine-tuning

Domain-adaptive pre-training and fine-tuning are common ways to transfer general language representations to specialised text [15, 18]. Full model adaptation is expensive for a large causal model, motivating parameter-efficient methods. LoRA freezes the base weights and introduces trainable low-rank update matrices [19]. QLoRA combines adapter learning with quantized base-model loading and provides the NF4 and memory-management techniques

that make 4-bit adaptation practical [8]. These papers motivate the compute-conscious method; they do not supply performance evidence for this particular model or corpus.

SkillLLM_v2 uses a Mistral Small 3.1 24B Instruct path, a 4-bit configuration, and LoRA settings stored in YAML. The Mistral model family established a widely used open-weight architecture line [21], while the project-specific training summary supplies the more important claim here: a recorded 80-epoch, two-process run with persisted adapter and evaluation artifacts. The thesis differentiates this implemented training path from the broad PEFT literature. It does not assert that all hardware settings or all configurations from the cited work were reproduced.

2.4 Taxonomy linking and retrieval

Taxonomies make a skill representation more useful for aggregation, reporting, and matching. ESCO provides a semantic-interoperability framing for job matching [27]; ESCOXLM-R and entity-linking work show how job-market language can be related to structured inventories [50, 51]. Large language models have also been investigated as zero-shot ESCO skill matchers [6]. A local implementation cannot be assumed to inherit the coverage or accuracy of those published systems. The audited Stage-2 code provides loaders and aliases for multiple sources, but its reported path is a custom alias-backed linker. Its 0.890 values are linked-candidate coverage, while a supervised canonical-link test set is absent.

Retrieval is relevant at two scopes. Dense passage retrieval [24], Sentence-BERT [39], SimCSE [11], and ColBERT [25] represent well-established semantic or late-interaction approaches. Retrieval-augmented generation combines retrieved evidence with a generator in knowledge-intensive tasks [30]. The project's 'ContextualRetriever' is deliberately much smaller: it scores lexical term overlap over contextualised records. It serves a deterministic smoke/demo path and an interface boundary for future dense retrieval. This distinction prevents the report from labelling a lexical overlap class as a complete RAG evaluation.

2.5 Graphs, matching, and contextual career intelligence

Career systems need relationships that simple extraction does not express: a user has a skill; a job requires it; a course teaches it; a missing skill explains a weak match. Skills2Job demonstrates graph-oriented job recommendation [12]. Person–job fit has also been studied

through domain-adapted deep networks and context-aware transformer matching [4, 31]. These studies motivate the graph and matching interfaces in SkillLLM_v2, but no paper can validate the repository's deterministic weights. The graph in this codebase is an in-memory NetworkX multigraph, and the matching result is an interpretable weighted formula.

Context management raises a different problem: sending an entire profile history with every query increases cost and can obscure fresh evidence. A compact, structured profile can cache stable, attributable facts while a retriever supplies dynamic job text. SkillLLM_v2 labels this separation CAG/RAG/graph policy: stable profile summary, aliases, and course summaries are cached; job postings and scraped descriptions remain retrieval targets; relational explanations remain in the graph. This is a concrete design and smoke-tested implementation, not an empirical comparison of cache-augmented generation, RAG, and GraphRAG methods.

2.6 Additional methodological perspectives

Soft-skill requirements are prominent but difficult to operationalise consistently in job text [40, 5]. Implicit-skill extraction also illustrates why a text mention should not be treated as a direct measure of competence [14]. SkillLLM_v2 consequently limits its extraction claim to explicit span evidence and reserves proficiency, transferability, and recency for caller-supplied profile fields or future evaluation.

Other NER lines make complementary trade-offs. Contextual string embeddings and pooled contextual embeddings show how character-level and document-context signals can affect entity boundaries [2, 1]; iterated dilated convolutions focused on fast recognition [42]. Broad surveys distinguish representation, decoder, data, and domain choices in deep NER [32]. These results motivate reporting the local BERT artifact as a control rather than as an exhaustive representation of token-classification approaches.

Parameter-efficient adaptation also includes prefix tuning and prompt tuning [33, 28]. Unified text-to-text modelling demonstrates the value of treating NLP outputs as controlled generation [38], while BART provides a denoising encoder–decoder perspective [29]. SkillLLM_v2 uses neither prefix tuning nor a BART-style architecture; these references clarify why the report calls its method causal-language-model SFT with LoRA rather than treating every structured-output model as equivalent.

The broader open-weight ecosystem includes mixture-of-experts Mistral-family work [22] and reproducibility-oriented fine-tuning tooling such as LlamaFactory [53]. Synthetic instruction construction is another relevant design direction [45]. The local Stage-2 experiment uses

the SkillSpan-derived JSON targets described in Chapter 5; it does not report a Mixtral comparison, LlamaFactory run, or synthetic-instruction ablation.

Document-link-aware pre-training, such as LinkBERT, provides another perspective on document relationships in language representations [46]. In contrast, the current project keeps retrieval, extraction, and graph paths modular: it does not train a link-aware language model or report end-to-end retrieval-grounded generation quality. This modularity is useful for testing because a retrieval error can be distinguished from a source-span error and from a graph-edge error.

The design also treats explanations and correction as part of research quality. Automated annotation studies show that language-model outputs can be useful but must be assessed against task-specific evidence and error patterns [13]. In SkillLLM_v2, a source span, offset, model source, and canonical attachment are retained so that a future user interface can expose and correct a recommendation's premises. This is a more modest and defensible use of automation than claiming that a generated profile is a verified employment assessment.

2.7 Research gap and positioning

The literature provides strong components: span extraction benchmarks, parameter-efficient adaptation, generalist NER, inventory linking, semantic retrieval, and graph recommendation. The local research gap is their operational composition. A sophisticated wrapper can destroy correct base spans through a boundary edit, a generic verifier rejection, or an aggressive merge. Conversely, a carefully scoped normalizer and profile builder can add useful downstream structure without claiming a span-F1 gain. SkillLLM_v2 is positioned as an evidence-preserving integration study. Its novel practical thesis is not "more modules are better"; it is that a primary extractor should be an explicit anchor and that every downstream alteration must be attributable, testable, and rejectable.

Chapter 3

Problem Formulation

3.1 Documents, spans, and labels

Let a document be a sequence of characters or tokens $X = (x_1, \ldots, x_n)$. A gold skill annotation is a tuple $s_i = (b_i, e_i, c_i)$, where b_i and e_i are start and exclusive end offsets into the original text and $c_i \in \{\textsc{Hard_Skill}, \textsc{Soft_Skill}\}$. A system predicts a set $\widehat{S} = \{\hat{s}_j\}_{j=1}^{m}$. A predicted span is exact when it has the same boundaries and label as one unmatched gold span. This definition makes the record's offsets part of the scientific object, not incidental metadata.

The implementation represents a prediction as a 'SkillCandidate' with the surface form, label, offsets, source, confidence, evidence, optional canonical identifier and name, and arbitrary metadata. The required fields make it possible to preserve the difference between a Mistral candidate, a GLiNER candidate, a dictionary candidate, and a merged candidate. A candidate with one missing offset is invalid; a candidate with both offsets absent is representable but cannot be counted as an aligned span. The primary evaluation path uses aligned predictions.

Hard skills are technical, tool, framework, domain, or method capabilities explicitly named in the input. Soft skills are interpersonal or behavioural capabilities explicitly named in the input. The restriction to explicit mentions is important. A job title, degree, company name, geographic location, or generic responsibility is not a skill merely because an observer could infer capability from it. The Mistral prompt repeats this restriction and asks for exact surface strings. It makes the system's intended epistemic boundary clear: extraction is evidence retrieval from a source, not latent ability assessment.

3.2 Metrics

For a set of predicted spans, let TP be the number of exact matches, FP the unmatched predictions, and FN the unmatched gold spans. Exact precision, recall, and F1 are

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}, \tag{3.1}$$

with zero denominators handled as zero by the implementation. The same calculation can be applied within the hard-skill and soft-skill subsets. Exact F1 is deliberately strict: a prediction that has the right words but includes an extra boundary token does not receive exact credit.

Partial-span F1 relaxes the matching condition to an overlap under the same label. It is informative when a model finds the semantic region but misses a modifier or delimiter. Token F1 measures overlap at token granularity and helps diagnose boundary behaviour. The evaluation artifacts also track parse failures and unaligned predictions. Together, these metrics distinguish three error classes that a single number would combine: failure to identify a skill, an almost-right boundary, and a generative output that cannot be grounded in the source.

The project reports canonical linking separately. For a set of candidates C, the stored coverage measure is

$$\text{coverage}(C) = \frac{|\{c \in C : c.\text{canonical_skill_id} \neq \varnothing\}|}{|C|}. \tag{3.2}$$

Coverage does not establish correctness. Linking accuracy would require gold canonical identifiers and is defined as the fraction of aligned predictions whose canonical identifier equals a corresponding gold identifier. Such labels are unavailable for the audited linking run. The report consequently uses “coverage” for the 0.890 figures and reserves “accuracy” for a future labelled evaluation.

3.3 Anchor-preserving composition

Let A denote the Raw Mistral anchor candidates and W a wrapper’s candidate output. Candidate merging is safe in a strong local sense when every $a \in A$ is retained unchanged, unless a later explicit output-selection policy chooses the anchor itself. In the implemented merger, a candidate that contains a Mistral source is retained. An unanchored candidate must satisfy source-dependent confidence conditions, alias evidence where configured, and

an anchor-overlap rule. This yields an add-only candidate policy rather than an unrestricted vote or replacement policy.

The wrapper-level policy is weaker but operationally useful. Let $f_e(\cdot)$ and $f_s(\cdot)$ denote exact-span and soft-skill F1, and let δ_e, δ_s be allowed drops. A wrapper is accepted when

$$f_e(W) \geq f_e(A) - \delta_e \quad \text{and} \quad f_s(W) \geq f_s(A) - \delta_s. \tag{3.3}$$

Otherwise the selected extraction output is A. The active safe-wrapper report records an exact F1 floor of 0.660035 and a soft F1 floor of 0.574046 for the persisted anchor. These values are configuration-specific safeguards, not universal statistical thresholds. An accepted wrapper can still be below a separately defined improvement target; this is why the report separates the `decision` and `improved_over_anchor` fields.

3.4 Canonical profile and downstream representations

An extracted surface form is not yet a durable user skill fact. The profile layer defines a fact

$$p = (u, s, \ell, q, E, M), \tag{3.4}$$

where u is a user identifier, s is a surface or canonical skill, ℓ is the hard/soft label, q is confidence, E is a list of evidence records, and M contains offsets and source metadata. Facts are grouped by canonical identifier when one exists and by normalized surface form otherwise. Repeated evidence can raise the stored maximum confidence, but evidence items are not discarded. Label disagreement is written as a conflict note rather than silently resolved.

The graph representation contains typed nodes for users, profiles, jobs, courses, skills, skill clusters, and occupations, with typed edges such as `HAS_SKILL`, `REQUIRES`, `PREFERS`, and `TEACHES`. A missing skill is a derived comparison between user skills and job requirements. The graph is intended to make the path behind a recommendation inspectable. It is not a learned graph embedding model in the audited code.

3.5 Deterministic matching and recommendation

The job matcher takes a profile skill set U, required skills R, preferred skills P, and optional externally supplied fit values. Let $r = |U \cap R| / \max(1, |R|)$, $p = |U \cap P| / \max(1, |P|)$, $c = |R \setminus U| / \max(1, |R|)$, and let s, e, u denote semantic-role, experience-level, and user-

preference fit fields. The source implements

$$\text{match} = \text{clip}_{[0,1]}(0.45r + 0.20p + 0.15s + 0.10e + 0.10u - 0.25c). \tag{3.5}$$

This formula is explanatory and deterministic. It should not be read as learned coefficients or as a calibrated probability of hiring.

For a course, the recommender scores gap coverage, prerequisite coverage, supplied level fit, supplied goal alignment, and a workload penalty with fixed weights 0.45, 0.25, 0.15, 0.15, and -1 respectively. The output explains which missing skills a course covers. There are no labelled job or course ranking results in the current artifact snapshot, so Equation (3.5) is a system specification rather than a validated decision rule.

Chapter 4

SkillLLM_v2 System Architecture

4.1 Architecture principle and implementation status

SkillLLM_v2 is a modular Python system under `SkillLLM_v2_stage2/src/skillllm_v2`. Its module layout separates extraction, verification, normalization, context, graph, matching, retrieval, training, evaluation, and pipeline concerns. This separation supports a central design principle: extraction evidence must remain inspectable even after a candidate is normalized or used by a downstream component. The project therefore makes provenance a data-model property rather than an after-the-fact report.

The architecture diagram in Figure 1.1 is intentionally status-aware. The green nodes correspond to source modules and persisted artifacts. Yellow nodes in later figures represent bounded experimental paths, such as wrapper variants with stored extraction metrics. Grey dashed nodes represent a future evaluation or production capability. This notation prevents an attractive diagram from turning a proposed GraphRAG-like deployment or an unevaluated ranking score into an implementation claim.

Table 4.1: Evidence status by subsystem. "Implemented" describes local source and artifacts, not a blanket production-readiness claim.

Subsystem	Status	Thesis-safe statement
Mistral extraction	Implemented, full-test metrics	Model-backed anchor; persisted predictions and metrics
BERT / GLiNER	Implemented baselines	BERT artifact is CPU-fast; GLiNER result is zero-shot
Candidate policy / gate	Implemented, full-test artifacts	Anchor-protection and tolerance gate evaluated on cached predictions
Taxonomy linking	Implemented, coverage measured	Attach-only local alias linking; no supervised linking-accuracy claim
Profile/context	Implemented, smoke-tested	Evidence-preserving structured packet and stable hash
Graph/matching/course	Implemented, smoke/demo-tested	Deterministic data flow; ranking quality not evaluated

4.2 Input contract and record schema

The extractor receives free text from a job posting, resume, user profile, or course description. It returns a list of records validated by `schema.py`. Required fields are `surface_form`, `label`, `start`, `end`, `source`, `confidence`, `evidence`, `canonical_skill_id`, `canonical_skill_name`, and `metadata`. The canonical fields may be null at extraction time, but their presence keeps a record shape stable across pipeline stages. Source is restricted to known extraction origins, and confidence must lie in $[0, 1]$. The schema also requires that offsets are either both known or both null and that a known start offset precedes its end.

This contract has two practical consequences. First, every later module can attach metadata without destroying a text-level audit trail. Second, a malformed output is rejected close to its origin. A wrapper cannot silently treat a text paraphrase as a source-grounded span if it lacks valid offsets. The `find_offsets` helper tries exact matching and then case-insensitive matching while avoiding reuse of already selected occurrences where possible. It is deterministic and simple; it does not perform semantic alignment. That limitation is appropriate for an exact-span benchmark because semantic recovery could conceal an output that does not actually reproduce the source string.

4.3 Mistral LoRA extraction anchor

The primary extractor is `MistralExtractor` in `extract/mistral_extractor.py`. It constructs a system message identifying the task as explicit skill extraction and a user message that requires valid JSON. The prompt explicitly prohibits paraphrase and unstated inference, asks for exact source strings, and distinguishes hard and soft skills. At inference, the model is called with sampling disabled. The parser accepts a raw JSON list/object, a fenced JSON block, or a decodable JSON fragment; otherwise it raises an error rather than guessing a structure.

Each parsed entry is converted into a `SkillCandidate`. `SKILL` labels are normalised to hard skill for a dataset interface that lacks a hard/soft distinction; unsupported labels are not retained as an invented class. A fixed confidence of 0.86 is assigned by this extraction adapter, so it must be understood as a pipeline score rather than calibrated probability. The implementation tracks latency and model-load errors in metadata. An alias-based fallback exists for smoke execution when the model path is unavailable and fallback is permitted. Crucially, the persisted primary Mistral metric JSON says that fallback was not used. The full-test result therefore rests on the configured model path and adapter, not this smoke convenience.

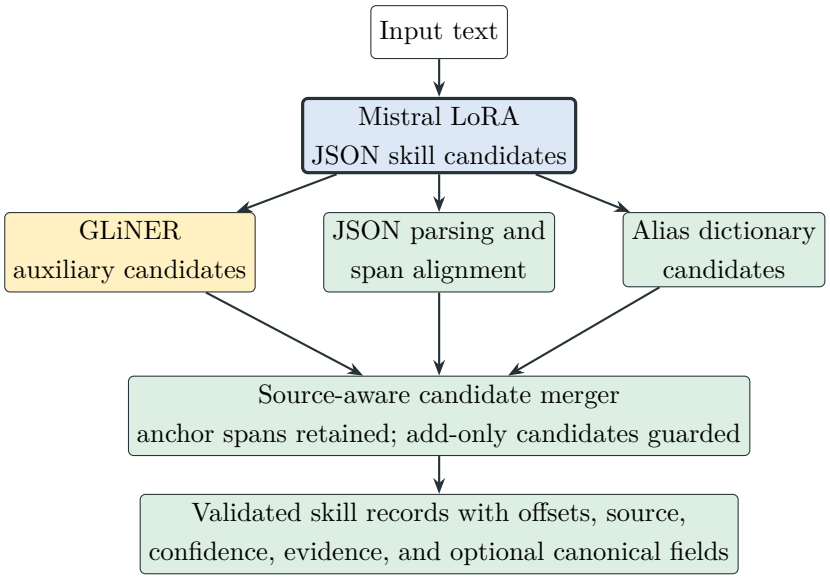


Figure 4.1: Implemented extraction path. The anchor creates source-grounded candidates; GLiNER and dictionary components can add candidates but do not replace the anchor by default.

4.4 Auxiliary extraction, merging, and refinement

GLiNER and the dictionary extractor have different roles. GLiNER can propose labels from a generalist NER model, while the dictionary extractor looks for aliases in taxonomy files and supports exact/fuzzy matching. Neither source is treated as automatically correct. `CandidateMerger` first validates every candidate, groups surface forms by normalised/alias-equivalent key and label, retains source provenance in `metadata.sources`, and removes redundant subsumed candidates. It then applies its add-only policy. An anchor candidate is recognised by a Mistral source; it is kept. A candidate without Mistral provenance must meet a configurable minimum confidence that depends on its source combination, and it may require exact alias evidence. Candidate spans overlapping an anchor span are withheld under the policy.

Boundary refinement, span verification, and confidence calibration are deliberately separate modules. The boundary refiner can preserve an original candidate or produce an annotated edit. Its tests assert that it preserves an unsubstantiated long span and never drops a candidate simply because a refinement is unavailable. The verifier applies deterministic checks and can reject generic material; the calibrator applies source- and label-sensitive thresholds. These are useful guardrails, but the source-level controls are not a substitute for empirical generalization. The attached span-verifier artifact shows that a model/dataset path exists, while the audit does not find a comprehensive learned-verifier outcome evaluation to report.

4.5 Safe-wrapper selection

The safe-wrapper entry point starts from cached 80-epoch Mistral predictions for the same 974-document evaluation path. It creates three variants: boundary refinement, Mistral plus GLiNER plus taxonomy, and a full path with merger, refiner, verifier, calibrator, and taxonomy linker. The non-regression gate then evaluates each candidate against anchor metrics and selects the candidate predictions only if exact-span and soft-skill floors are met. If either guardrail fails, the selected extraction predictions are the untouched anchor predictions even though linked candidates can still be produced for downstream inspection.

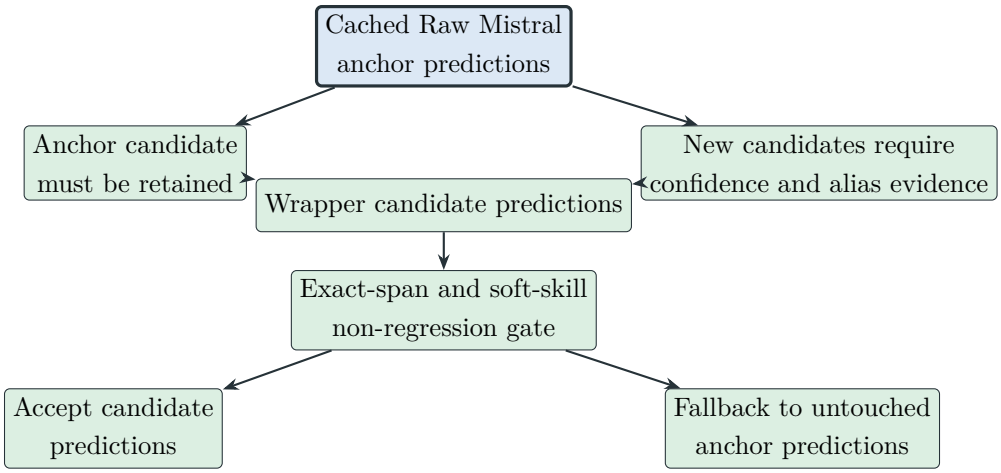


Figure 4.2: Safe-wrapper design. Protection occurs twice: candidate merging preserves Mistral-originated spans, and the evaluation gate can fall back to cached anchor predictions.

The gate is an output-selection mechanism, not a proof of causal improvement. In the persisted safe-wrapper artifact, all three variants receive `accept`, but their `improved_over_anchor` fields are false because none reaches the historical target of 0.675034 exact F1. This is a valuable distinction for a system report. A quality floor can be a reasonable deployment safeguard, while an improvement claim needs stronger evidence such as repeated paired runs, confidence intervals, or a pre-registered evaluation protocol.

4.6 Canonicalization and taxonomy attachment

`TaxonomyLinker` turns a surface form into an optional canonical identifier and name. The alias infrastructure builds an index from configured records; the analysis report identifies 696 canonical entries in the current table. The linker can record alias, fuzzy, and BM25-like decisions. Its essential architectural rule is attach-only behaviour: re-linking never changes `surface_form`, `start`, `end`, `label`, or `source`. This makes canonicalization orthogonal to extraction scoring. A later matching layer may use the canonical ID, but an evaluator can always recover the original text and source.

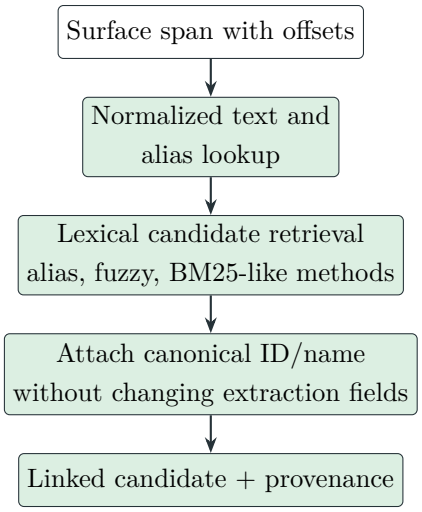


Figure 4.3: Attach-only normalization. The canonical fields enrich a candidate but do not rewrite the extraction fields used by span evaluation.

The implementation includes loaders named for ESCO, O*NET, Lightcast, and custom records. The reported configuration points at local custom aliases. It is therefore precise to call this an extensible taxonomy interface with a local alias-backed evaluation path. It is imprecise to call it a complete production import of every external taxonomy. The distinction is reinforced by the linking reports: high coverage is available, but a gold canonical-ID dataset for accuracy is not.

4.7 Context, graph, and downstream services

The `context` package transforms already extracted skills into a `ProfileSummary`. Each skill fact carries a surface form, hard/soft label, optional canonical fields, maximum confidence, a list of evidence items, and source offsets. Grouping uses a canonical identifier where present and normalised surface form otherwise. When labels conflict, the summary writes a conflict note. It neither infers a hidden capability nor resolves a contradiction through an opaque model. `SkillMemory` groups facts by label and constructs an alias index. The cached-context builder serialises a packet and stores a SHA-256 content hash, enabling a direct equality check for identical inputs.

The graph layer uses `networkx.MultiDiGraph`. It has typed User, Profile, Job, Course, Skill, SkillCluster, and Occupation nodes. Its direct retrieval functions list a user's skills, a job's required skills, and courses that teach a skill; the explainer attempts a shortest path. The `JobMatcher`, `GapDetector`, and `CourseRecommender` consume simple dictionaries and expose their components in output records. This creates a transparent vertical slice from

extracted spans to recommendations, but it is intentionally not presented as a deployed service, learned retriever, or empirically validated career adviser.

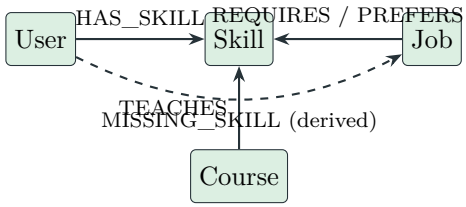


Figure 4.4: Implemented in-memory knowledge-graph schema. A missing skill is derived from a user/job comparison; it is not an asserted source fact.

4.8 Architecture summary

The completed architecture has one authoritative extraction record and multiple carefully constrained enrichments. At no point does the diagram require an unverified model to overwrite a source span. This makes the system suitable for iterative research: new linkers, retrievers, graph stores, or learned rankers can be introduced behind stable record contracts and evaluated separately. The same modularity also exposes the current boundary: only the extraction and wrapper metrics have full-test numeric evidence; the rest of the architecture must be judged as implemented design and smoke-tested data flow until labelled downstream evaluation is added.

Chapter 5

Dataset and Experimental Methodology

5.1 Evidence-first methodology

The methodology follows a strict evidence hierarchy. A quantitative statement in this report must come from a persisted metric artifact, a generated report derived from such an artifact, a configuration file, a training summary, or a local data file. A source module establishes that an interface or algorithm is implemented, but it does not establish its performance. A status marker establishes that an automated stage finished, but it does not establish that the result is a full-scale experiment. This hierarchy is necessary because the repository contains full-test outputs, smoke tests, demo records, historical analyses, and an initially uncommitted working tree.

The audit written before this report is the operational version of this method. It reconciles the later safe-wrapper artifacts with an earlier wrapper-error report, identifies the primary result files, and records unavailable downstream metrics as ‘NA‘. The result is more conservative than treating every number found in the directory as interchangeable, but it makes the final argument reproducible and falsifiable.

5.2 Dataset and local representation

The primary corpus is the local export at `data/source/skillspan_hf`. Its README attributes SkillSpan to Zhang et al. [48], specifies CC BY 4.0 licensing, and identifies the English job-posting sources as StackOverflow/technical and STAR/house. Each JSON line

has an `idx` document delimiter, token list, `tags_skill`, `tags_knowledge`, and source. The tag names should not be equated mechanically with the report's hard/soft labels without inspecting the data-preparation path; the project creates its internal typed span representation from the local export.

Table 5.1: Local SkillSpan export used by the Stage-2 pipeline. The local JSON line counts match the dataset README.

Split	Sentences	Use in this report
Train	4,800	Supervised fine-tuning examples
Development	3,174	Persisted training evaluation and development split
Test	3,569	Local corpus test partition; the reported cached Mistral evaluation path contains 974 documents

The total local row count is 11,543. This is smaller than a naive reading of the paper's 14.5K sentence description only if one mistakes rows or pre-processing states for the original corpus. The thesis reports both facts as separate: the original paper's corpus description and the actual local split counts consumed by this pipeline. It does not invent a reconciliation that is absent from repository artifacts.

5.3 SFT data preparation and prompt contract

`build_mistral_sft_data.py` iterates over train, development, and test splits and writes a JSONL record with four elements: an identifier, an instruction, original input text, and an output object holding gold skills. The training renderer then produces a Mistral chat sequence consisting of a system instruction, a user request containing the source text, and an assistant JSON target. The target strips offsets and retains text plus label. At evaluation time, offsets are re-established by source-string alignment. This choice is significant: training teaches structured string production, whereas evaluation insists that the predicted string maps back to an observed substring.

The prompt requests only explicit mentions and asks for JSON with a `skills` array containing `text` and `label` values. It permits hard, soft, and generic `SKILL` labels at the interface, with generic values mapped to hard skill in the extractor. No free-form explanation is requested from the model, which narrows the decoding surface and makes parse errors

observable. The prompt can reduce certain formatting failures; it does not guarantee schema adherence or semantic correctness. The parser and validator remain necessary.

5.4 Training procedure

The full configuration is `configs/skillllm_v2_training_80ep_2gpu.yaml`. It specifies seed 42 and an 80-epoch full run. `mistral_lora_trainer.py` loads the model locally, selects bfloat16 when supported and float16 otherwise, configures 4-bit NF4 loading when enabled, prepares the model for k-bit training, enables gradient checkpointing, and injects LoRA adapters into the listed projection modules. The trainer uses a causal language-model objective over rendered chat text, a cosine scheduler, 5% warmup by default, an 8-bit paged AdamW optimizer in the 4-bit path, and no external reporting backend.

Table 5.2: Verified Mistral LoRA settings from `skillllm_v2_training_80ep_2gpu.yaml` and the persisted training summary.

Setting	Value
Seed	42
Backbone	Mistral Small 3.1 24B Instruct
Adaptation	LoRA; rank 16, alpha 32, dropout 0.05
Quantization	4-bit loading; NF4 path in source
Epochs	80
Batch / accumulation	1 / 32
Learning rate	0.00012
Maximum sequence length	512
Parallelism	2-process DDP, GPUs 0 and 1
Recorded runtime	324,217.825 seconds

The persisted training summary records a two-process world size, 4,800 training samples, 3,174 development samples, epoch 80, a training loss of 0.143082, a development loss of 0.397449, and 324,217.825 seconds of training runtime. The research-refresh manifest records two RTX 3090 GPUs, PyTorch 2.6.0+cu124, and Python 3.12.3. These records make the compute account concrete. They do not make loss directly comparable with other projects, prove that the final epoch is globally optimal, or quantify seed sensitivity.

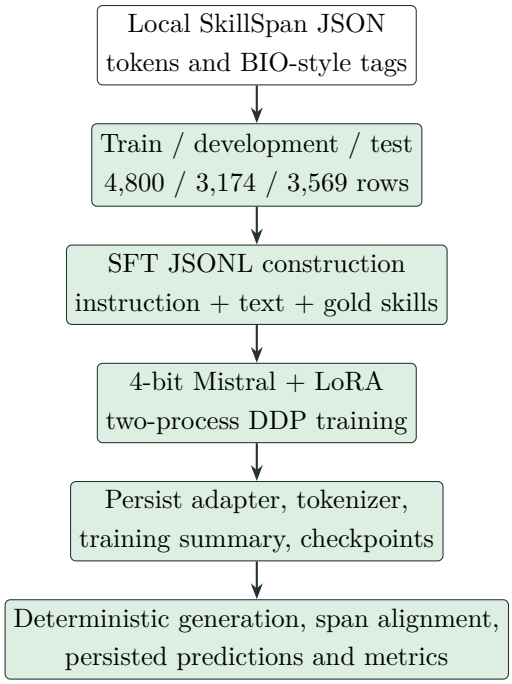


Figure 5.1: Training and evaluation pipeline traced to the local data-preparation and trainer modules. The persisted evaluation artifact is distinct from smoke-mode fallback.

5.5 Baselines and comparison discipline

The project contains a BERT baseline and a GLiNER baseline. The BERT metric file identifies itself as a CPU-fast token-classification baseline. The GLiNER metric file states that fine-tuning was skipped and that the run is zero-shot/few-shot. Both are useful reference points because they use the same local evaluation tooling and show the magnitude of the Mistral result in the repository. They are not intended as exhaustive tuned baselines. In particular, their lower results cannot support a general conclusion that all encoder models or all generalist NER systems are ineffective for skill extraction.

The report also distinguishes legacy Phase-A and Stage-2 evidence. A root-level thesis evidence report references earlier multi-LLM benchmarks with different epoch counts and configurations. They have research value, but the present numeric comparison table is restricted to Stage-2 persisted artifacts. This avoids combining a three-epoch result, a 80-epoch result, a smoke test, and a demo under one apparent benchmark protocol.

5.6 Extraction evaluation

The extraction evaluator compares gold and predicted spans and reports exact span, partial span, token, hard-skill, soft-skill, macro, and micro metrics. Exact matching uses label and boundary agreement. Partial matching and token F1 provide diagnostic context when a system locates a relevant phrase but selects a different boundary. The primary metric is exact-span F1 because downstream canonicalization is only meaningful when it can be traced to a source span. Parse failure and alignment information further distinguish a model that has no candidate from one that emits ungrounded text.

For wrapper experiments, the methodology is paired: cached anchor predictions and candidate wrapper predictions are evaluated on the same 974-document path. The safe-wrapper script records candidate and selected metrics, guardrail floors, deltas, a decision, and an explanation. This is stronger than comparing unrelated aggregate runs, yet still not a multi-seed statistical study. It identifies whether a particular wrapper snapshot is compatible with the anchor; it does not estimate a population-level treatment effect.

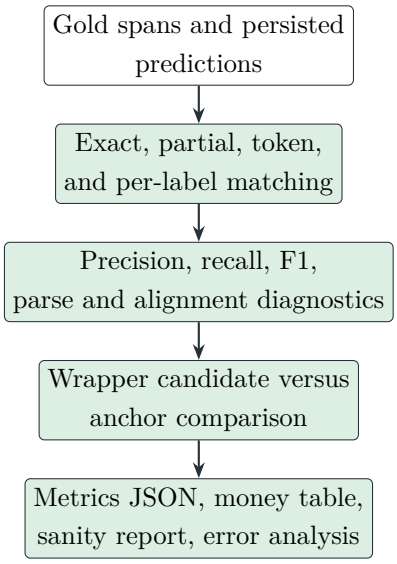


Figure 5.2: Evaluation path for extraction and wrapper selection. Downstream ranking metrics use a separate, currently unavailable labelled protocol.

5.7 Normalization and downstream methodology

Linking evaluation is deliberately separated from extraction. The taxonomy error-analysis report evaluates how often candidate records obtain canonical attachments from a 696-entry local alias table and breaks methods down into BM25, alias, fuzzy, and none. Because no

gold canonical IDs are supplied for a supervised reranker, the reported 0.890 values are coverage measures. The `linking_eval.csv` file has `taxonomy_linking_accuracy,0.0`; it must not be overwritten with coverage in a final table.

The graph, job matching, gap detection, and course recommendation modules are exercised through unit tests and demo artifacts. Their output formats include matched skills, missing skills, evidence strings, recommendation reasons, and courses that cover gaps. Neither `matching_eval.csv` nor `course_recommendation_eval.csv` has a usable label-derived score: nDCG@10 and P@5 are both 'NA'. Accordingly, this report analyses their data flow and formulas, not their real-world efficacy.

5.8 Reproducibility procedure

The implementation produces a layered trail: configuration leads to adapter/checkpoint and training-summary artifacts; generation leads to predictions, raw outputs, and metrics; wrapper processing leads to candidate and linked records; reports summarise the results. `prediction_provenance_check.py` verifies that selected downstream files are newer than the upstream Mistral predictions and rejects a metric note indicating fallback use. The report states that no stale or missing downstream artifacts were detected. Timestamps are useful safeguards but are not a replacement for versioned Git history, immutable artifact storage, checksums of every input, or environment lock files. Those items are future reproducibility work.

Chapter 6

Experiments

6.1 Experimental objectives

The experiments are organised around a narrow claim: identify the strongest extraction anchor in the persisted Stage-2 artifacts and test whether modular additions remain compatible with it. This differs from a broad model sweep. The repository contains earlier multi-model work, but the current Stage-2 study is Mistral-focused and uses stored artifacts from an 80-epoch training path. The comparisons answer three practical questions. First, can the adapted causal model produce source-aligned hard and soft skill spans substantially better than the included BERT and zero-shot GLiNER reference artifacts? Second, do boundary, auxiliary-candidate, taxonomy, and full-wrapper paths alter the anchor metrics? Third, are the resulting records usable by the implemented profile, graph, matching, and recommendation paths without converting smoke success into a ranking claim?

All reported numeric comparisons are descriptive measurements from the persisted files. The work does not report confidence intervals, significance tests, or multi-seed variation because those artifacts are absent. A difference smaller than a thousandth should be read as an observation in this evaluation snapshot, not as a robust superiority statement.

6.2 Experiment E1: extraction baselines

E1 compares the BERT baseline, GLiNER baseline, and Raw Mistral LoRA anchor. The BERT file reports exact-span F1 0.059150, token F1 0.274922, hard-skill F1 0.129799, and soft-skill F1 0.018796. Its note calls it a CPU-fast token-classification baseline. The GLiNER file reports exact-span F1 0.075833, token F1 0.114409, hard-skill F1 0.054559, and soft-skill

F1 0.096571. It explicitly identifies the run as zero-shot because fine-tuning was skipped. These labels constrain interpretation: E1 compares available local reference implementations, not fully optimized replications of the BERT or GLiNER literature.

The Mistral anchor was trained with the settings in Table 5.2. Its result is evaluated through deterministic JSON generation, parsing, substring alignment, and the extraction evaluator. Its artifact says 'Model load fallback used: False', which is a crucial validity condition because the extractor contains a deterministic smoke fallback for unavailable model files. The Mistral value is therefore the main model result in this report.

6.3 Experiment E2: historical wrapper diagnosis

E2 is a historical diagnostic, not a row in the primary result table. `wrapper_error_analysis.md` reports an anchor of 0.675034 exact F1 and 0.609792 soft F1 on a 974-document test path. It reports that an older boundary-refiner variant fell to approximately 0.5704 exact F1 and 0.4258 soft F1, while an older full CAG/graph path fell to approximately 0.5616 exact F1 and 0.4080 soft F1. The diagnosis attributes 194 boundary-related regressions to the boundary refiner in the full path and 29 further cases to verifier false rejection. It is a valuable negative result: a post-processor can reduce a model's exactness even when its stated purpose is to clean up spans.

The audit found later full-test artifacts created after this analysis. Their anchor has 0.670069 exact F1 rather than 0.675034, and their safe-wrapper code uses anchor-protection and a gate. These runs should not be averaged or represented as a single ablation series. E2 explains the engineering response; E3 provides the current metric snapshot.

6.4 Experiment E3: safe-wrapper variants

E3 loads the cached full-test Mistral predictions and evaluates three wrapper variants on the same document path. The first applies a non-destructive boundary refiner with verifier and calibrator. The second combines Mistral, GLiNER, and dictionary candidates, applies an anchor-protected merger, then attaches taxonomy information. The third combines the merger, boundary refiner, verifier, calibrator, and taxonomy linker in the full CAG/graph path. The script writes both candidate predictions and selected predictions so that a fallback decision is inspectable.

The active guardrails compare exact-span and soft-skill metrics with the anchor. A candidate can be accepted while still failing to reach the historical improvement target. This

is intentional. The policy defines a deployment-like safety threshold, whereas the improvement target remains an informational goal. In the stored output, all variants are accepted because each clears exact and soft floors. The Mistral+GLiNER+taxonomy candidate is 0.000007 lower in exact F1 and 0.001741 lower in soft F1 than the anchor; the full candidate is 0.001033 higher in exact F1 but 0.000291 lower in soft F1. Such deltas should be treated as essentially a compatibility result, not a reason to prefer one architecture on accuracy alone.

Table 6.1: Safe-wrapper gate results. Acceptance means that the candidate clears configured non-regression floors; it does not mean that it reached the historical improvement target.

Variant	Anchor exact	Candidate exact	Anchor soft	Candidate soft	Decision
Boundary refiner	0.670069	0.671111	0.604046	0.605505	accept
GLiNER + taxonomy	0.670069	0.670062	0.604046	0.602305	accept
Full CAG/graph	0.670069	0.671102	0.604046	0.603755	accept

6.5 Experiment E4: taxonomy attachment

E4 evaluates whether the local linker attaches canonical fields to wrapper candidates. The analysis report states that an earlier four-entry fallback was replaced by a 696-entry alias table. For the Mistral+GLiNER+taxonomy path, it reports total linking coverage 0.882139 under a detailed analysis and 0.890271 in the metric artifact; for the full path it reports 0.838908 in the detailed analysis and 0.890374 in the metric artifact. These values reflect different candidate artifacts/stages and should not be collapsed. The primary table uses the metric-artifact value, while the detailed report remains useful for error categories and unlinked surface forms.

Most importantly, no canonical gold labels are available for a supervised reranker. The project reports method counts for alias, fuzzy, BM25, and no link, but cannot establish that every attachment is correct. E4 therefore supports an engineering claim—the normalizer attaches canonical metadata with broad local coverage—not a semantic-accuracy claim. The attach-only constraint ensures that a wrong link does not alter the original span metrics, though it can still harm a downstream recommendation and must be evaluated later.

6.6 Experiment E5: profile, graph, and recommendation vertical slice

E5 is an integration and smoke experiment. The profile summarizer aggregates extracted skills, their evidence, and their offsets. The cached-context builder separates stable profile memory, dynamic job-posting retrieval policy, and graph-path policy, then computes a content hash. The graph builder creates typed nodes and direct relationships; the matcher, gap detector, and recommender emit inspectable JSON objects. Unit tests verify a representative user–skill–job–course flow, and demo artifacts contain a profile, job matches, skill gaps, course recommendations, and an explanation.

The evidence supports a vertical-slice claim: records can travel from extraction into a structured profile and then into explainable downstream objects. It does not support a utility claim. The matching-evaluation CSV records job nDCG@10 as ‘NA‘, and the course-recommendation evaluation CSV records P@5 as ‘NA‘, because relevance labels are unavailable. No human-subject study, click-through test, or offline ranking benchmark is stored. Thus E5 evaluates execution and traceability, not recommendation quality.

6.7 Artifact tracking and protocol fidelity

Figure 6.1 shows the experimental artifact path. The project writes model-training summaries, model files, predictions, raw outputs, metrics, gate reports, money tables, and error analyses under conventional artifact/report directories. The provenance-check script compares downstream modification times with Mistral predictions and checks the metrics note for fallback use. The persisted report indicates no stale or missing artifacts. This check prevents a common research-reporting failure in which a downstream table is silently generated from an older anchor output.

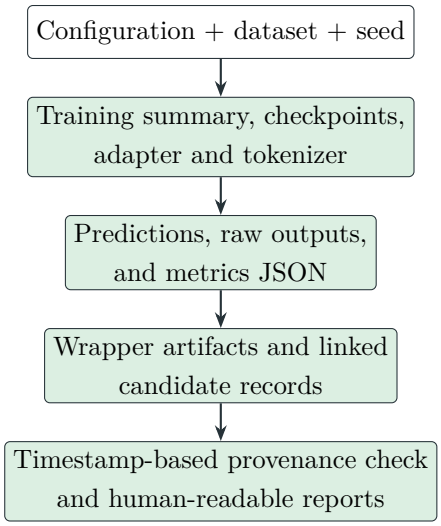


Figure 6.1: Artifact and provenance pipeline. The timestamp check is a practical safeguard, not an immutable experiment-management system.

The remaining protocol limits are clear. The Stage-2 tree has no commit, a model-selection criterion beyond the stored output is not documented as a pre-registration, the full evaluation path contains 974 documents despite the local test JSON having 3,569 rows, and multi-seed reruns are absent. Future experiments should version every source and configuration, explain the 974-document evaluation subset in a machine-readable manifest, and predefine their primary score and selection rule.

Chapter 7

Results

7.1 Primary extraction results

Table 7.1 presents the thesis's primary numerical result table. It is generated from the persisted Stage-2 metric artifacts and agrees with the metric-sanity and money-table reports. The Raw Mistral LoRA path is the clear anchor within this collection: exact-span F1 is 0.670069, token F1 0.787886, hard-skill F1 0.726450, and soft-skill F1 0.604046. The result is substantially larger than the stored BERT and zero-shot GLiNER scores. The appropriate conclusion is local and bounded: under this repository's stored evaluator, training path, and artifact set, the adapted Mistral extractor is the best available extraction component.

Table 7.1: Primary full-test extraction artifacts. Values are copied from persisted `metrics.json` files, not inferred from prose reports.

System	Exact F1	Token F1	Hard F1	Soft F1
BERT baseline	0.059150	0.274922	0.129799	0.018796
GLiNER baseline (zero-shot)	0.075833	0.114409	0.054559	0.096571
Raw Mistral LoRA anchor	0.670069	0.787886	0.726450	0.604046
Mistral + GLiNER + taxonomy	0.670062	0.787575	0.728020	0.602305
Full CAG/graph wrapper	0.671102	0.787575	0.728618	0.603755

The contrast between exact and token F1 is informative. For the Mistral anchor, token F1 exceeds exact F1 by about 0.118. This pattern is expected when some predicted regions overlap gold spans but have non-identical boundaries. It reinforces why a report on a downstream career system must not use a token-only measure: the original surface span is later used for evidence display and canonicalization. The hard/soft difference is also material.

Soft-skill F1 is about 0.122 lower than hard-skill F1 for the anchor, leaving a clear target for future annotation analysis and model improvement.

The BERT and GLiNER measurements should not be misread as a literature leaderboard. BERT is identified as a CPU-fast baseline, and GLiNER is zero-shot/few-shot. Nonetheless, they are useful controls because they reveal that the result is not a minor score variation around a shared baseline. They also illustrate a methodological lesson: merely inserting a general NER component does not establish suitability for typed skill spans under an exact evaluation interface.

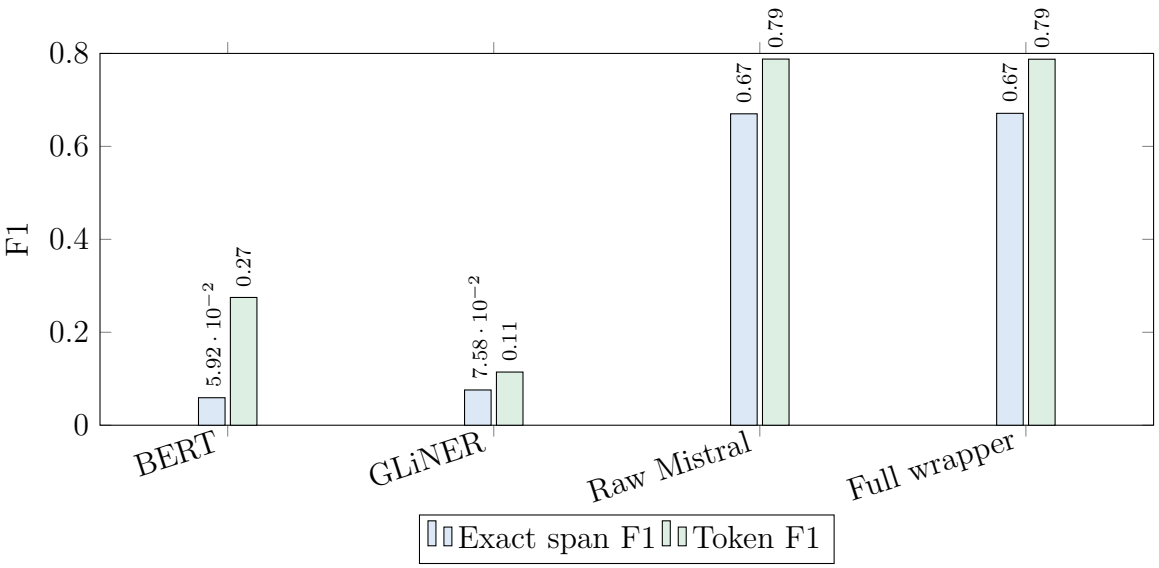


Figure 7.1: Exact-span and token F1 from primary persisted artifacts. The full wrapper's exact-span difference from Raw Mistral is 0.001033 and should not be interpreted as a robust gain.

7.2 Wrapper results and selection semantics

The wrapper variants preserve the headline result rather than transform it. The boundary-safe variant records 0.671111 exact F1 and 0.605505 soft F1. The Mistral+GLiNER+taxonomy variant records 0.670062 and 0.602305. The full CAG/graph variant records 0.671102 and 0.603755. Each selected result passes the gate's configured floors, and each is marked 'accept'. The full wrapper has the largest exact F1 observed among these files, but its advantage over the Mistral anchor is only 0.001033. Its soft-skill F1 is slightly lower than the anchor's. The evidence supports the wording "accepted as non-regressive under configured tolerances"; it does not support "wrapper improves extraction."

This result is scientifically useful rather than disappointing. It means that a modular pipeline can enrich extraction with source, canonicalization, context, and graph metadata

without a large measured loss in the stored evaluation. It also warns against using wrapper complexity as a substitute for a strong primary model. A consumer of the output can treat Mistral as the provenance source for extraction and treat the wrapper as a guarded enrichment layer. The architecture remains valuable even in the absence of a large F1 increase because downstream systems need IDs, evidence, and explanations, but their value must be evaluated on their own metrics.

7.3 Linking and downstream results

The full CAG/graph metrics file records `linking_accuracy` 0.890374, while the detailed linking report calls the same kind of quantity coverage and states that supervised reranker labels are unavailable. The safer term is therefore *linking coverage*: a large share of candidate records obtained a canonical attachment from the local alias system. It cannot be called canonical-link accuracy without a gold target. The code's attach-only property protects extraction F1 from the addition of a canonical field, but it does not establish the correctness of all attached IDs.

The downstream result status is even more constrained. The full wrapper's metric file contains null values for job nDCG@10 and course precision@5, and their report CSV files write 'NA'. The code produces job-match scores, gap types, recommended courses, and explanations. The integration slice is therefore available for qualitative inspection and future annotation, but no quantitative claim is made about relevance, fairness, usefulness, or hiring outcomes.

7.4 Answer to the research questions

RQ1 is answered by the Mistral artifact: the primary model obtains 0.670069 exact-span F1 and 0.604046 soft-skill F1 in the stored Stage-2 evaluation. RQ2 is answered descriptively: the stored BERT and GLiNER baselines are much lower, and the wrapper variants remain near the anchor rather than delivering a large improvement. RQ3 is answered by the code and gate artifacts: candidate source protection, add-only admission, attach-only linking, and fallback are concrete mechanisms that prevent uncontrolled post-processing.

RQ4 is answered at implementation level. The profile/context path preserves extraction evidence and offsets, handles duplicate keys, writes conflicts, separates stable and dynamic context, and hashes a cached packet. RQ5 is answered by an explicit boundary: typed graph, matching, gap, and course modules exist and are smoke-tested, but no labelled nDCG or P@5

evaluation is available. The most important result is therefore a systems result: extraction quality is an invariant that the rest of the architecture must preserve, rather than a quantity downstream components can freely trade away for apparent sophistication.

Chapter 8

Error Analysis and Non-Regressive Design

8.1 Why an error analysis is necessary

Aggregate F1 can conceal the mechanism of a regression. A wrapper may improve the appearance of one candidate while removing many correct anchor spans; a taxonomy linker may attach a plausible identifier to a wrong surface form; a verifier may reject uncommon technical terms as generic language. Because SkillLLM_v2 is meant to support later explanation, the source of a failure matters as much as its score. The repository contains an unusually informative historical wrapper report and a later safe-wrapper evaluation. Read together, they show why the architecture protects anchor spans and records a selection decision.

8.2 Historical destructive post-processing

The historical `wrapper_error_analysis.md` is not merged into the primary metric table, but it is a strong diagnostic. It reports an anchor exact-span F1 of 0.675034, token F1 of 0.797798, hard-skill F1 of 0.731138, and soft-skill F1 of 0.609792. Its older boundary-refiner path reports approximately 0.5704 exact F1 and 0.4258 soft F1; the older full CAG/graph path reports approximately 0.5616 exact F1 and 0.4080 soft F1. The direction is unambiguous in that historical snapshot: the post-processors reduced extraction quality sharply.

The report attributes 194 full-wrapper regressions to bad boundary edits, two to bad label changes, and 29 to verifier false rejection. It records 225 total full-wrapper regressions. The boundary refiner alone had 194 regressions. These counts identify a root cause more useful

than a generic statement that "the wrapper underperformed." A high-quality extractor had already produced a boundary that the downstream rule changed incorrectly. The fix is not a more aggressive rule; it is to retain the anchor unless a correction can be justified and to make the correction observable in metadata.

Table 8.1: Historical error diagnosis from `wrapper_error_analysis.md`. These values describe a prior wrapper analysis and are not pooled with the newer primary table.

Stage	Exact F1	Soft F1	Exact gains	Exact losses
Raw Mistral anchor	0.675034	0.609792	–	–
Boundary refiner	0.5704	0.4258	53	194
Mistral + GLiNER + taxonomy	0.6515	0.5777	36	0
Full CAG/graph	0.5616	0.4080	78	225

The apparent zero regressions in the historical Mistral+GLiNER+taxonomy row do not make it a universal winner; its exact and soft values remain below that historical anchor. They instead suggest that additions and edits have distinct risk profiles. Add-only candidate generation can increase recall without deleting a base prediction, while boundary modification and generic rejection can damage previously correct spans. This observation is implementation-specific and requires renewed testing on every code/configuration change.

8.3 From diagnosis to control points

The current code introduces four control points. First, a candidate with Mistral provenance is anchor-protected in 'CandidateMerger'. Second, non-anchor candidates must satisfy confidence and optional alias-evidence requirements before they can be added. Third, the boundary-refiner tests explicitly require preservation when taxonomy evidence is absent and prohibit candidate disappearance. Fourth, the non-regression gate evaluates a candidate wrapper output against exact and soft floors and can select cached anchor predictions instead.

These controls address different failure modes. Source protection stops a merger from deleting an anchor candidate before evaluation. Add-only filtering limits false-positive additions. Boundary preservation stops a local rule from turning uncertainty into an edit. The gate protects the final output even when an earlier module changes behaviour. None of these mechanisms proves semantic correctness. They are layered safeguards with traceable failure modes, which is more appropriate than trusting a complex wrapper as a monolith.

8.4 Current safe-wrapper interpretation

The later safe-wrapper run yields a materially different outcome: all three variants clear the chosen non-regression floors. This shows that the corrected path is at least compatible with the persisted anchor under the recorded evaluation. It does not invalidate the historical diagnosis. Rather, the diagnosis motivated a redesign, and the new run is evidence that the redesign avoided the large measured collapses observed earlier. The experiment is still a single stored snapshot. A new model checkpoint, alias table, or boundary rule could cause another regression and should trigger the same paired evaluation.

The report's honest language matters here. Calling the safe wrapper "non-regressive" is correct only relative to its configured floors and artifact path. Calling it "better" because one exact F1 value is 0.001033 higher would overstate a difference with no repeated runs or uncertainty estimates. Similarly, a wrapper that falls back to the anchor would not be a failed system: it would have exercised the intended safety mechanism and preserved the primary extraction output.

8.5 Likely remaining error categories

Several error categories remain relevant even where a count is not available. First, the model can produce a correct concept with a string that does not occur verbatim, causing an alignment failure. Second, the same surface string can occur more than once; deterministic occurrence selection may attach a prediction to the wrong mention. Third, hard/soft labels can be ambiguous for terms such as collaboration, communication, project management, or domain-specific practices. Fourth, the local alias table can leave emerging technologies, abbreviations, or malformed source text unlinked. The linking report's unlinked examples include technical names and compressed forms such as ‘k8s‘, ‘yaml‘, and ‘sonarqube‘, showing the practical value of coverage analysis.

Finally, downstream errors can originate upstream even when downstream code is deterministic. A false negative extraction becomes a missing skill, which may lower a match score or cause a course to be recommended unnecessarily. A wrong canonical ID can create a false graph edge. An explicit evidence field, source offset, and explanation string allow a user or researcher to trace such a decision back to a candidate record. This is why provenance is not merely documentation: it is a debugging interface.

8.6 Error-analysis protocol for future runs

Future experiments should preserve the current paired structure and extend it. For each candidate wrapper, save anchor predictions, candidate predictions, selected predictions, and a row-level action log. Stratify errors by hard/soft label, source domain, span length, alias availability, parser outcome, and edit type. Add a blind manual sample review so that a score change can be associated with interpretable examples. Most importantly, keep the extraction anchor immutable during wrapper evaluation. If a new primary model is trained, establish its own artifact, then repeat wrapper evaluation from that version rather than reusing earlier thresholds without verification.

Chapter 9

Context Intelligence: Profiles, Graphs, Matching, and Recommendations

9.1 From a span list to a durable profile

Span extraction is intentionally local: given a sentence or document, it returns the capability mentions supported by that text. A career-facing system needs an additional representation that can accumulate evidence across a resume, user-confirmed entries, target roles, completed courses, and interactions. Passing every raw input to a model on every request is inefficient and makes contradictions difficult to inspect. Conversely, replacing the history with an opaque summary risks erasing the source of a claim. SkillLLM_v2 implements a narrow middle ground: a structured, evidence-preserving profile summary.

`ProfileSkillFact` contains a surface form, a hard/soft label, optional canonical identifier and name, a confidence, a list of evidence records, and metadata. Each evidence record contains a source, text, confidence, and timestamp. `summarize_profile` receives a raw profile text and explicitly supplied extraction results; it does not infer a hidden skill. It groups records by canonical identifier if available, otherwise by normalised surface form. When duplicate evidence arrives, it keeps evidence items and takes the maximum confidence rather than silently averaging away conflicting support. Source offsets are preserved in metadata.

This design gives a later UI or LLM concrete answers to questions such as "Why does the profile contain Python?" and "Which input mentioned this skill?" It also has a modest conflict mechanism. If the same normalised name arrives under different labels, a conflict note is recorded. It does not yet model temporal recency, evidence reliability by source, proficiency levels, or semantic contradiction. For example, it has no first-class representation of a claim that a user stopped using a tool or has only introductory experience. Those are

intended future extensions, not current implicit behaviour.

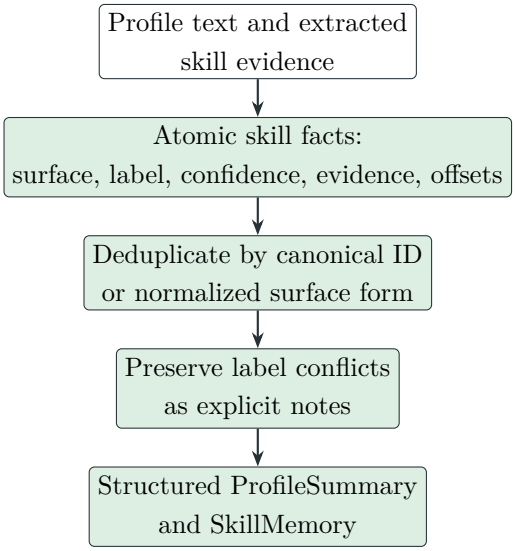


Figure 9.1: Implemented evidence-preserving profile summarization. It aggregates explicit candidates and records conflicts instead of inferring or silently resolving them.

9.2 Context compaction policy

The context-compaction function returns three intentionally separate sections. Its stable cache contains a profile summary, skill memory, taxonomy aliases, and optional course-catalog summaries. Its dynamic-retrieval policy identifies job postings and scraped descriptions as sources to retrieve rather than cache. Its graph policy describes user-to-skill-to-job-to-course paths that belong in a relational layer. The cached packet carries the three policies, a packet type, and a SHA-256 hash of the serialised payload.

The terms CAG and RAG are used in this report to describe local context-management roles, not to claim a benchmarked implementation of a particular published CAG/GraphRAG algorithm. The cached portion is stable in the sense of input role, not permanently true; an application must invalidate or recreate it when a user updates their profile. The dynamic policy is a declaration that job records change frequently, not a production web-crawling system. The retrieval class currently performs deterministic lexical overlap over contextualised records. It creates a testable interface but does not claim dense semantic retrieval.

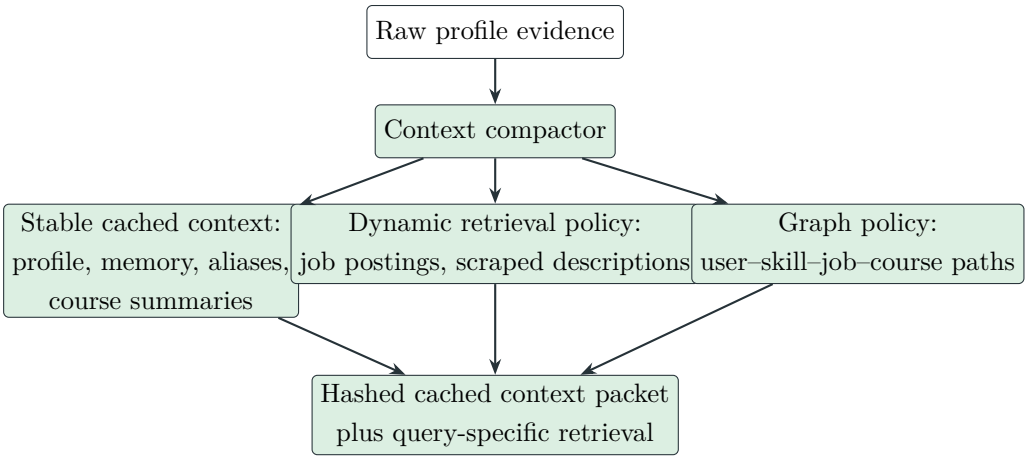


Figure 9.2: Implemented context-compaction policy. Stable profile structures are cached; dynamic job information and graph explanations remain separate responsibilities.

9.3 Why evidence preservation matters for compaction

An evidence-preserving compact profile has three system benefits. First, it makes source attribution available after deduplication. A canonical `amazon_web_services` fact can retain both an "AWS" resume mention and an "Amazon Web Services" job-history mention rather than forcing one surface form to win. Second, it supports constrained updates. A future skill-confidence model can append a new evidence event or revise an aggregation function without rewriting the original extraction. Third, it makes a contextual packet inspectable: the hash detects changes to serialised content, and the separated policy fields reveal why a fact was cached or retrieved.

This approach also reduces a common overclaim in career systems. A user profile is not a ground-truth competency graph merely because it is structured. It is a set of claims with source and confidence. The current profile data type does not determine mastery, duration, recency, or employer validation. A recommendation generated from it should expose that it is based on extracted textual evidence and should invite correction. The existing code's refusal to infer hidden skills is therefore a useful privacy and reliability property.

9.4 Knowledge graph representation

The graph subsystem uses NetworkX to create an in-memory `MultiDiGraph`. It supports User, Profile, Job, Course, Skill, SkillCluster, and Occupation nodes. Its edge vocabulary includes `HAS_SKILL`, `MISSING_SKILL`, `REQUIRES`, `PREFERS`, `TEACHES`, `REQUIRES_PREREQUISITE`,

`SIMILAR_TO`, and `BELONGS_TO`. The public builder methods create user-skill, job-requirement, and course-skill relationships. The retriever lists direct neighbours, while the explainer computes a shortest path if one exists and returns a textual arrow-separated explanation.

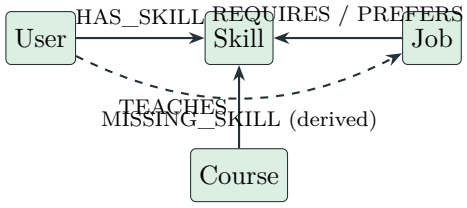


Figure 9.3: Skill graph relation types used by the current vertical slice. Direct edges are constructed from supplied records; a missing skill is derived.

The graph is useful even before learned graph reasoning. A deterministic graph prevents a downstream explanation from being detached from the skills and course records it used. It allows a path such as user → known skill → job requirement → course to be surfaced for inspection. Still, the present graph does not persist to a database, resolve real-world duplicate entities beyond the linker, learn node embeddings, or evaluate graph retrieval against relevance labels. It should be described as a lightweight graph representation and explanation mechanism.

9.5 Job matching and skill-gap detection

`JobMatcher` consumes a profile with a `skills` list and a job with `required_skills` and `preferred_skills`. It computes overlap, missing required skills, optional supplied semantic-role fit, experience-level fit, user-preference fit, and a penalty proportional to missing required skills. The fixed equation in Equation (3.5) is implemented directly in source. The returned record includes a clipped score, matched skills, missing skills, critical missing skills, evidence strings showing coverage, and a short recommendation reason.

`GapDetector` expands the missing-skill view. For a job requirement, a skill can be missing-critical, weak, outdated, uncertain, or covered by an explicitly supplied transferable mapping. Preferred but absent skills form a separate missing-preferred category. This vocabulary is useful for transparency because an absent string and a low-confidence/obsolete skill are not the same intervention. Yet the classification depends on supplied profile fields; the system does not currently estimate proficiency or obsolescence from raw text.

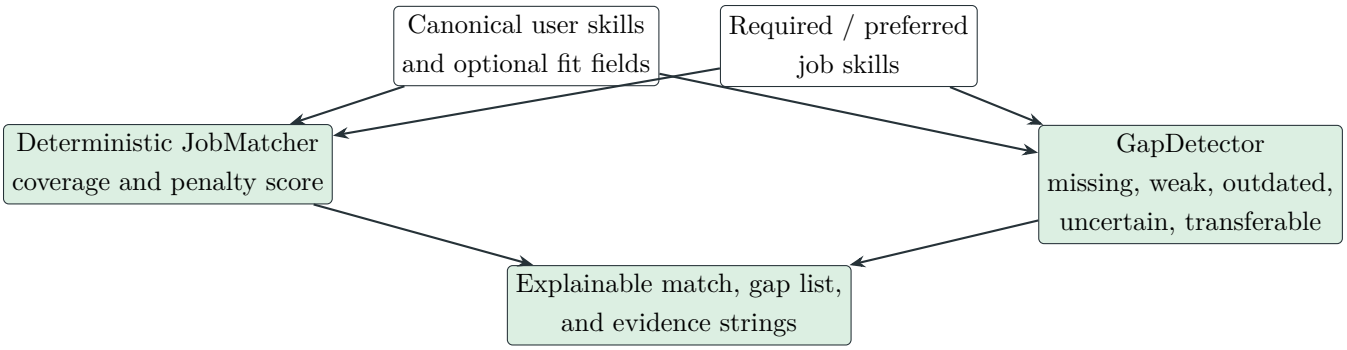


Figure 9.4: Implemented deterministic matching and gap-detection flow. Optional semantic, experience, and preference fit fields are caller-supplied rather than learned by the repository.

The score must not be treated as a hiring prediction. Its weights are transparent implementation choices, not fitted parameters. The lack of labelled relevance data is especially important here: a high coverage score could still recommend a role that mismatches seniority, location, compensation, accessibility needs, or nuanced work experience. A responsible system would use this output as evidence-backed decision support and make the components visible, not as automatic eligibility screening.

9.6 Course recommendation

`CourseRecommender` accepts gap objects, a course catalogue, and an optional profile. It considers only courses that cover at least one gap. For each it calculates gap coverage, prerequisite fit, supplied level fit, supplied goal alignment, and workload penalty, then clips a weighted sum to $[0, 1]$. The output includes course identifier/name, covered skills, gap-coverage fraction, a score, and a sentence explaining the covered gaps. This is a readable implementation of a rule-based baseline, and its unit test verifies a simple expected course.

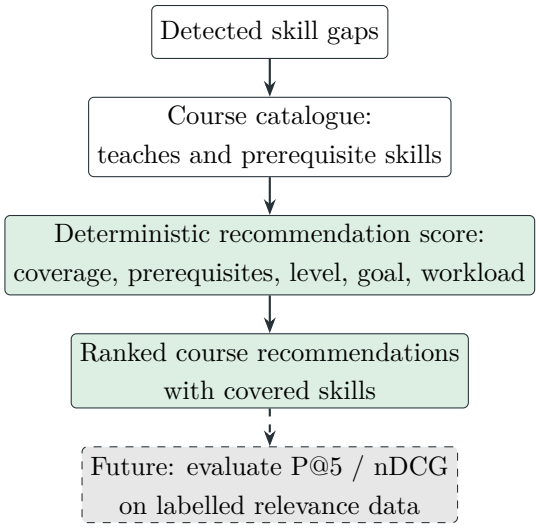


Figure 9.5: Implemented course-recommendation flow and its explicit evaluation boundary. P@5 is unavailable because no labelled course relevance data are persisted.

The current catalogue inputs are demo data. No thesis claim is made about course quality, learner success, fairness, or P@5. A meaningful next evaluation would assemble user/job/course triples with independently judged relevant courses, ensure train/test separation where learned components are added, compare simple coverage ranking with stronger retrieval baselines, and analyse whether recommendations differ systematically across user groups. Until then, the recommender is an inspectable vertical-slice component rather than a validated intervention.

9.7 A staged deployment interpretation

The context and graph architecture suggests a safe future rollout. Stage one can expose extraction and canonicalization with evidence and correction affordances. Stage two can offer profile compaction, showing exactly what will be cached and allowing the user to add, remove, or dispute facts. Stage three can show deterministic gap and course suggestions as explanations, clearly labelled as unvalidated. Only after a labelled offline evaluation and a fairness/privacy review should the system use learned rankers or consequential matching decisions. This staged interpretation aligns the strength of a feature's claims with the strength of its evidence.

Chapter 10

Limitations, Threats to Validity, and Future Work

10.1 Claim boundary

This report is intentionally narrower than the codebase's conceptual scope. Its strongest quantitative evidence concerns span extraction and three wrapper variants. Its strongest engineering evidence concerns explicit schemas, source-preserving operations, unit tests, configuration, and persisted artifacts. The graph, profile, matching, and recommendation modules are implemented but do not have labelled effectiveness measures. Figure 10.1 summarises this boundary. A reader should treat it as part of the result, not as a caveat to skip.

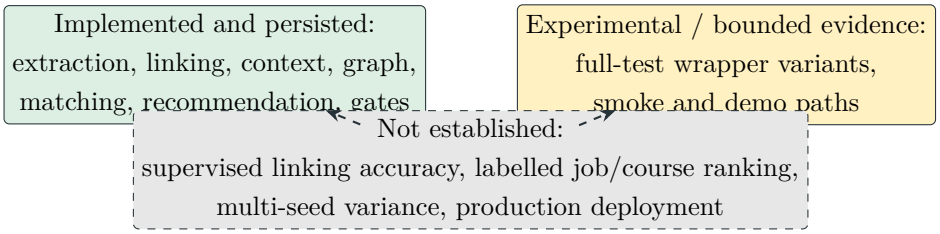


Figure 10.1: Evidence status across SkillLLM_v2. A complete vertical slice is not the same as a complete outcome evaluation.

10.2 Dataset and evaluation limitations

SkillSpan is an English job-posting benchmark with expert hard/soft span annotation [48]. It is an appropriate starting point, but a strong score on this export does not establish transfer to resumes, course catalogues, conversational profile updates, or multilingual labour markets.

The local data contain technical and house sources, and the selected training/evaluation transformation may differ from other published SkillSpan protocols. The full Mistral artifact reports a 974-document cached evaluation path although the local test JSON has 3,569 rows. The repository does not provide a sufficiently explicit manifest explaining that subset in this audit. That ambiguity is a reproducibility limitation and should be resolved before a final defence claim.

Exact span F1 is rigorous but incomplete. It penalises a boundary difference even when a human would consider the prediction useful, while partial and token metrics may make a boundary failure look less serious than it is for evidence display. No qualitative, blinded annotation study is stored to determine whether errors are concentrated in rare technologies, compositional skills, soft skills, source domains, or annotation ambiguities. The report also lacks confidence intervals and multi-seed runs. The observed 0.001-level wrapper differences could change under another seed or subset.

10.3 Model and training limitations

The persisted Mistral run is substantial: it uses an 80-epoch 4-bit LoRA configuration and two RTX 3090 GPUs. Nevertheless, it is a single run with stored losses and artifacts, not a hyperparameter or checkpoint-selection study. The reports do not establish a validation-selected epoch, compare LoRA ranks, test prompt variants, or provide a full ablation of quantization and distributed settings. The extractor has a deterministic smoke fallback; although the primary metric says fallback was false, every future run must preserve that check. A model-loading issue could otherwise turn a seemingly valid model result into a keyword baseline.

The Mistral adapter assigns a fixed candidate confidence in its extraction adapter. The confidence calibrator uses rules and thresholds, but no calibration curve, expected calibration error, or probability-quality assessment is persisted. Confidence should therefore be read as a gating/control signal, not as a statistically calibrated belief. The same caveat applies to verifier outputs and matcher scores.

10.4 Normalization and ontology limitations

The local alias table has 696 canonical entries and produces high attachment coverage in the stored wrapper reports. Coverage is not correctness. Gold canonical labels, a validated taxonomy version, hierarchy-aware scoring, and error analysis by taxonomy source are required

before making a canonical-link accuracy claim. The code has loaders for ESCO, O*NET, Lightcast, and custom data, but the reported setup uses custom aliases. External taxonomy licenses, versions, semantic drift, and alias collisions would need governance in a production setting.

Attach-only normalization is deliberately conservative, but it creates a trade-off. It protects extraction metrics from a bad link, while an incorrect canonical field can still mislead matching, aggregation, or graph paths. A future linker should keep candidate alternatives, calibration/evidence scores, taxonomy provenance, and user correction history rather than pretending that one alias match is definitive.

10.5 Context, graph, and recommendation limitations

The profile summary preserves supplied evidence but does not model recency, proficiency, evidence reliability, temporal contradiction, or user consent beyond the input interface. It must not be populated from sensitive personal attributes or used to infer protected characteristics. The cache hash detects a payload change but does not solve cache invalidation, retention policy, access control, encryption, deletion requests, or cross-device synchronisation. The current CAG/RAG separation is a software policy rather than a measured reduction in token use, latency, hallucination, or retrieval error.

The graph is in memory and its relation vocabulary is manually constructed. It does not provide transactional updates, scalable storage, graph embeddings, learned edge confidence, or a formal provenance model for derived edges. The matcher and course recommender use hand-specified weights. They have no nDCG, P@5, calibration, fairness, or user-satisfaction result. A job match score must never be used as an automated hiring, admission, or eligibility decision. Any serious deployment would require human oversight, an appeal/correction route, data minimisation, and a fairness evaluation that is appropriate to the intended population.

10.6 Reproducibility and report-format limitations

The Stage-2 branch has no commits. The artifact directory preserves useful evidence but cannot by itself establish exactly which source revision created every output. Timestamp-based provenance checking is helpful but vulnerable to copied files and does not hash all dependencies. The environment description is partial: Python, PyTorch/CUDA, hardware, and YAML values are available, while a complete lock file, container, dataset fingerprint, and immutable experiment registry are not. The report's new figures are reproducible TikZ

source, but the final institutional format still needs the author, official course designation, committee, and template confirmation. It is not a substitute for SFSU Graduate Studies approval.

10.7 Future work

The most valuable next step is a versioned, labelled evaluation program rather than another unmeasured architectural module. First, commit the Stage-2 source, record dataset hashes and the exact 974-document evaluation manifest, and rerun the primary experiment across multiple seeds. Report mean, dispersion, and paired wrapper tests. Second, create a canonical-link dataset with gold IDs and evaluate both coverage and accuracy; preserve alternate candidates and test long-tail aliases. Third, collect job/profile and course/gap relevance labels, define privacy and consent rules, and evaluate matching and recommendation with nDCG, P@5, coverage, calibration, and subgroup error analysis.

On the modelling side, investigate soft-skill errors by span length, annotation source, and semantic category. Compare the fixed prompt with controlled variants, examine calibration, and conduct a proper boundary-refiner ablation that logs every edit. On the systems side, extend profile facts with temporal evidence and user-confirmed proficiency while preserving the original sources; introduce a versioned graph store and explicit derived-edge provenance; and replace the lexical retrieval demo with evaluated sparse, dense, and hybrid baselines. Each addition should remain behind the same anchor-preserving and artifact-provenance controls that produced the central finding of this report.

Chapter 11

Conclusion

SkillLLM_v2 demonstrates a complete, evidence-aware path from span-level skill extraction to canonicalized profile, contextual packet, graph, gap, and course-recommendation records. Its principal measured result is the persisted 80-epoch Mistral LoRA extractor: exact-span F1 0.670069, token F1 0.787886, hard-skill F1 0.726450, and soft-skill F1 0.604046 in the audited Stage-2 artifact. The included BERT and zero-shot GLiNER artifacts are far lower under the same local reporting path. Within this repository, the adapted Mistral model is consequently the correct extraction anchor.

The system's more important design result is negative and constructive at once. Historical wrapper analysis showed that boundary edits and verifier rejection could substantially damage correct anchor spans. The current architecture responds with Mistral-source protection, add-only auxiliary candidates, attach-only linking, and a non-regression gate with explicit fallback. The later safe-wrapper artifacts remain close to the anchor and are accepted under their configured floors, but their thousandth-scale metric differences do not establish a general accuracy gain. The report therefore rejects the premise that more downstream components automatically make extraction better.

The remaining modules are useful because they enrich rather than replace evidence. A canonical identifier can be attached without changing the original span. A profile fact can retain its source, offset, confidence, and conflict note. A cached packet can separate stable information from dynamic job text. A graph and deterministic score can make a gap or course recommendation inspectable. These features are implemented and tested as a vertical slice. They are not yet evidence that the system matches jobs or recommends courses effectively, fairly, or safely.

Next work should version inputs, repeat extraction across seeds, and add gold labels. Until then, extraction provenance remains the central invariant.

Appendix A

Reproducibility Record

This appendix records the local paths and conditions needed to inspect or reproduce the claims in this report. It is intentionally not a promise that every command will run unchanged on another machine. The source tree has no commit at the audited branch, large model files are local, and the report has no container or lock-file artifact. A future reproducibility release should address those gaps.

A.1 Environment evidence

The run manifest at `SkillLLM_v2_stage2/reports/skillllm_v2/research_refresh/run_manifest.json` records Linux, Python 3.12.3, PyTorch 2.6.0+cu124, CUDA availability, and two NVIDIA GeForce RTX 3090 devices with 24 GiB memory each. The training summary at `artifacts/skillllm_v2/mistral_lora/model/training_summary.json` records world size 2 and epoch 80. The active full-run YAML is `configs/skillllm_v2_training_80ep_2gpu.yaml`. These artifacts describe the recorded run; any new run should write a fresh manifest rather than inheriting this metadata.

The package list in `requirements.txt` names PyTorch, LLaMA-Factory, GLiNER, NetworkX, scikit-learn, YAML, and related libraries. It is not a complete platform lock. In particular, the exact CUDA driver, compiler, PyPI resolution, installed Git dependency commits, tokenizer assets, and model download state should be captured in a future environment manifest.

A.2 Data and source checkpoints

The dataset root is `data/source/skillspan_hf/`. Its three JSON files have 4,800, 3,174, and 3,569 lines for train, development, and test. The dataset README contains its attribution and licence. Before a rerun, record a SHA-256 fingerprint of each JSON file and a manifest identifying the 974-document evaluation path used by the cached Mistral artifacts. This report does not reconstruct that subset because the current artifacts do not explain it sufficiently.

The Mistral training code is `src/skillllm_v2/train/mistral_lora_trainer.py`; SFT data construction is `src/skillllm_v2/train/build_mistral_sft_data.py`. The primary adapter/checkpoint assets are stored under `artifacts/skillllm_v2/mistral_lora/model/`. `training_summary.json` identifies the model path as `models/mistral-small-3.1-24b-instruct-2503`. This report never modifies any of those model or experiment artifacts.

A.3 Representative commands

The repository's shell scripts provide the authoritative launch interfaces. The following commands are representative inspection and validation commands; they should be run from the Stage-2 root after activating a compatible project environment and setting 'PYTHONPATH=src'.

```
cd "/home/923873155/Sem 4 Research/SkillLLM_v2_stage2"
export PYTHONPATH=src
export CUDA_VISIBLE_DEVICES=0,1
export TOKENIZERS_PARALLELISM=false
export WANDB_DISABLED=true

pytest -q tests/skillllm_v2 tests/test_skillllm_v2_stage2.py
python -m skillllm_v2.eval.prediction_provenance_check
python -m skillllm_v2.eval.run_safe_wrapper \
  --config configs/skillllm_v2_training_80ep_2gpu.yaml
```

The final command re-evaluates wrapper components against cached anchor predictions; it does not retrain Mistral. Training is consequential GPU work and should be launched only through the project scripts/configuration after confirming model paths, storage, and a fresh experiment ID. The report's metrics are the persisted historical artifacts, not outputs generated while typesetting this document.

A.4 Artifact verification sequence

The following sequence maintains the report's claim boundary:

1. Inspect the configuration, data paths, and model/training summary.
2. Confirm that primary Mistral 'metrics.json' says fallback was false.
3. Read the primary prediction and metric artifacts before generating tables.
4. Run the provenance check and preserve its output with the report revision.
5. Treat any status file ending in or describing a smoke operation as smoke evidence only.
6. Regenerate wrapper metrics on the same manifest before comparing a code change with the anchor.
7. Version source, configuration, dataset fingerprints, and generated tables together before external release.

A.5 Tests and their interpretation

The focused test suite covers schemas, Mistral SFT-data construction, BERT/GLiNER smoke paths, candidate merging, boundary refinement, calibration, taxonomy linking, profile summarisation, cached context, graph construction, job matching, course recommendation, pipeline gates, and non-regression selection. A current report-verification run passed 50 tests in 1.34 seconds under Python 3.13.12. That runtime is separate from the recorded training environment. A passing unit test demonstrates an invariant or small example; it does not replace the full-test extraction metrics, linking labels, ranking labels, or a deployment evaluation. The distinction should remain visible in automated continuous integration as well as in the thesis.

Appendix B

Prompt and Data-Contract Details

B.1 Extraction prompt design

The Mistral extractor uses a system message stating that it identifies explicit skill spans from resumes, job postings, and professional text and must return valid JSON only. The user prompt then directs the model to extract exact surface spans, forbid paraphrase and unstated inference, include technical and soft skills only when explicitly mentioned, ignore titles and generic responsibilities unless explicitly skills, and use an empty list when no skills occur. It asks for objects with ‘text‘ and ‘label‘ fields.

The training renderer constructs the assistant target from the local gold spans as compact JSON. It uses the text and label but not the offset in the target. This means that source alignment is not delegated to the model output schema: after generation, `find_offsets` locates the predicted text in the input and the validator confirms a coherent candidate. The design is conservative. A model output that paraphrases a true skill is not silently credited as an exact span.

B.2 Skill candidate contract

The following abstract JSON record mirrors `SkillCandidate.to_dict()` and its validator. It is a representation schema, not a record copied from a user.

```
{
  "surface_form": "Python",
  "label": "HARD_SKILL",
  "start": 18,
```

```
  "end": 24,
  "source": "mistral",
  "confidence": 0.86,
  "evidence": "experience with Python and SQL",
  "canonical_skill_id": "optional-id-or-null",
  "canonical_skill_name": "optional-name-or-null",
  "metadata": {
    "raw_label": "HARD_SKILL"
  }
}
```

The validator restricts labels to `HARD_SKILL` and `SOFT_SKILL`, source to `bert`, `mistral`, `gliner`, or `dictionary`, confidence to the inclusive interval $[0, 1]$, and offsets to a valid pair. A merger can add `metadata.sources` to show that several candidates support the same retained record. A linker can fill canonical fields. Neither operation should erase the original source information.

B.3 Profile and context contract

`ProfileSummary` stores a user identifier, generation time, an ordered list of skill facts, target roles, missing skills, course history, preferences, and conflict notes. A fact stores a confidence and one or more `Evidence` items. The cached context packet supplies packet type, cache policy, payload, and content-hash fields. Its policy contains three lists:

```
cag:   profile_summary, skill_memory, taxonomy_aliases, course_catalog_summary
rag:   job_postings, scraped_descriptions
graph: user_skill_job_course_paths
```

This schema states which category an item belongs to; it does not by itself implement database persistence, authorization, or automatic invalidation. Applications consuming it should surface user controls and treat the hash as an integrity/debugging aid rather than a security boundary.

B.4 Matching and recommendation output contract

The job matcher returns a job identifier, a score in $[0, 1]$, matched skills, missing skills, critical missing skills, evidence strings, and a short explanation label. The gap detector emits

a list of skill/gap-type pairs. The course recommender returns a course identifier/name, skills it covers, a gap-coverage ratio, a score, and a reason. These outputs contain transparent intermediate values, which is preferable to a bare score in a research prototype. They should not be interpreted as a decision on employment eligibility or a statement of a user's competence.

Appendix C

Audit Trail and Artifact Index

C.1 Primary source of truth

The detailed audit is maintained outside the PDF at `docs/thesis/codebase_audit.md`. It describes the branch state, source modules, configurations, data counts, persisted metrics, historical conflict, test coverage, and claim boundary. This appendix provides a compact index so that a committee member can locate the evidence behind each main result without relying on an informal narrative.

Table C.1: Claim-to-artifact index.

Claim	Primary local evidence	Interpretation limit
Mistral extraction F1	`artifacts/skillllm_v2/mistral_lora/metrics.json`	One persisted evaluation snapshot; no multi-seed estimate
Mistral training	`mistral_lora/model/training_summary.json`	Records runtime/loss/world size, not global optimality
BERT and GLiNER baselines	Their respective `metrics.json` files	CPU-fast and zero-shot qualifications apply
Safe-wrapper decision	`safe_wrapper/metrics.json` and `safe_wrapper_gate.md`	Tolerance-based compatibility, not statistical superiority
Historical regression	`wrapper_error_analysis.md`	Older diagnostic; not pooled with later table

Claim	Primary local evidence	Interpretation limit
Linking coverage	`mistral_gliner_taxonomy/metrics.json`; `full_cag_graph/metrics.json`	Coverage is not supervised linking accuracy
Profile/context path	`context/*.json`; context source/tests	Smoke-tested structured data flow
Graph/matching/course path	`demo/*.json`; source/tests	nDCG@10 and P@5 unavailable
Artifact ordering	`prediction_provenance_check.md`	Timestamp check, not immutable lineage
Hardware/runtime	`research_refresh/run_manifest.json`	Describes recorded environment only

C.2 Known metric conflict

Two source families contain different anchor values. The historical error report uses 0.675034 exact F1 and 0.609792 soft F1 for an earlier anchor. The later primary Mistral ‘metrics.json‘ uses 0.670069 and 0.604046. This report uses the later persisted metrics for the main table because they are the artifacts selected by the current money table, sanity report, safe-wrapper metrics, and provenance check. It retains the earlier report only to explain the failure mode that motivated source-aware policy and gate design. No average, difference, or ranking in this thesis mixes the two source families.

C.3 Report maintenance checklist

Before submitting a revision, regenerate tables from the current metric artifacts; rebuild the PDF; update the audit with any changed path or metric; rerun the garbage-text scan; verify every figure/table reference; and update the administrative cover-page details only after the department confirms the final route and template. If a new model run changes the anchor, regenerate every wrapper comparison rather than copying the previous threshold. If labelled downstream data become available, add a separate evaluation chapter/table rather than rewriting the current smoke/demo claims.

Bibliography

[1] Alan Akbik, Tanja Bergmann, and Roland Vollgraf. Pooled contextualized embeddings for named entity recognition. 2019.

[2] Alan Akbik, Duncan Blythe, and Roland Vollgraf. Contextual string embeddings for sequence labeling. 2018.

[3] Akshay Bhola, Kishaloy Halder, Animesh Prasad, and Min-Yen Kan. Retrieving skills from job descriptions: A language model based extreme multi-label classification framework. 2020.

[4] Shuqing Bian, Wayne Xin Zhao, Yang Song, Tao Zhang, and Ji-Rong Wen. Domain adaptation for person-job fit with transferable deep global match network. 2019.

[5] Federica Calanca, Luiza Sayfullina, Lara Minkus, Claudia Wagner, and Eric Malmi. Responsible team players wanted: an analysis of soft skill requirements in job advertisements. 2019.

[6] Benjamin Clavié and Guillaume Soulié. Large language models as batteries-included zero-shot ESCO skills matchers. 2023.

[7] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. 2020.

[8] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. 2023.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. 2019.

[10] Jinlan Fu, Xuanjing Huang, and Pengfei Liu. SpanNER: Named entity re-/recognition as span prediction. 2021.

[11] Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. 2021.

[12] Anna Giabelli, Lorenzo Malandri, Fabio Mercorio, Mario Mezzanzanica, and Andrea Seveso. Skills2Job: A recommender system that encodes job offer embeddings on graph databases. 2021.

[13] Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. ChatGPT outperforms crowd-workers for text-annotation tasks. 2023.

[14] Akshay Gugnani and Hemant Misra. Implicit skills extraction using document embedding and its use in job recommendation. 2020.

[15] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. 2020.

[16] Amirhossein Herandi, Yitao Li, Zhanlin Liu, Ximin Hu, and Xiao Cai. Skill-LLM: Repurposing general-purpose LLMs for skill extraction. *arXiv preprint arXiv:2410.12052*, 2024.

[17] Phuong Hoang, Thomas Mahoney, Faizan Javed, and Matt McNair. Large-scale occupational skills normalization for online recruitment. 2018.

[18] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. 2018.

[19] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. 2022.

[20] Zhiheng Huang, Wei Xu, and Kai Yu. Bidirectional LSTM-CRF models for sequence tagging. 2015.

[21] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. 2023.

[22] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian

Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. 2024.

[23] Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S. Weld, Luke Zettlemoyer, and Omer Levy. SpanBERT: Improving pre-training by representing and predicting spans. 2020.

[24] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. 2020.

[25] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. 2020.

[26] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. 2016.

[27] Martin le Vrang, Agis Papantoniou, Erika Pauwels, Pieter Fannes, Dimitri Vandensteen, and Jan De Smedt. ESCO: Boosting job matching in europe with semantic interoperability. 2014.

[28] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. 2021.

[29] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. 2020.

[30] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. 2020.

[31] Changmao Li, Elaine Fisher, Rebecca Thomas, Steve Pittard, Vicki Hertzberg, and Jinho D. Choi. Competence-level prediction and resume and job description matching using context-aware transformer models. 2020.

[32] Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li. A survey on deep learning for named entity recognition. 2022.

[33] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. 2021.

[34] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. 2019.

[35] Xuezhe Ma and Eduard Hovy. End-to-end sequence labeling via bi-directional LSTM-CNNs-CRF. 2016.

[36] Antoine Magron, Anna Dai, Mike Zhang, Syrielle Montariol, and Antoine Bosselut. JobSkape: A framework for generating synthetic job postings to enhance skill matching. 2024.

[37] Khanh Nguyen, Mike Zhang, Syrielle Montariol, and Antoine Bosselut. Rethinking skill extraction in the job market domain using large language models. 2024.

[38] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2020.

[39] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-Networks. 2019.

[40] Luiza Sayfullina, Eric Malmi, and Juho Kannala. Learning representations for soft skill matching. 2018.

[41] Elena Senger, Mike Zhang, Rob van der Goot, and Barbara Plank. Deep learning-based computational job market analysis: A survey on skill extraction and classification from job postings. In *Proceedings of NLP4HR*, 2024.

[42] Emma Strubell, Patrick Verga, David Belanger, and Andrew McCallum. Fast and accurate entity recognition with iterated dilated convolutions. 2017.

[43] David Wadden, Ulme Wennberg, Yi Luan, and Hannaneh Hajishirzi. Entity, relation, and event extraction with contextualized span representations. 2019.

[44] Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, and Guoyin Wang. GPT-NER: Named entity recognition via large language models. 2023.

[45] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. 2023.

[46] Michihiro Yasunaga, Jure Leskovec, and Percy Liang. LinkBERT: Pretraining language models with document links. 2022.

[47] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. GLiNER: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of NAACL*, 2024.

[48] Mike Zhang, Kristian Jensen, Sif Sonniks, and Barbara Plank. SkillSpan: Hard and soft skill extraction from English job postings. In *Proceedings of NAACL-HLT*, pages 4962–4984, 2022.

[49] Mike Zhang, Rob van der Goot, Min-Yen Kan, and Barbara Plank. NNOSE: Nearest neighbor occupational skill extraction. 2024.

[50] Mike Zhang, Rob van der Goot, and Barbara Plank. ESCOXLM-R: Multilingual taxonomy-driven pre-training for the job market domain. 2023.

[51] Mike Zhang, Rob van der Goot, and Barbara Plank. Entity linking in the job market domain. 2024.

[52] Meng Zhao, Faizan Javed, Ferosh Jacob, and Matt McNair. SKILL: A system for skill identification and normalization. In *Proceedings of AAAI*, 2015.

[53] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyan Luo. LlamaFactory: Unified efficient fine-tuning of 100+ language models. 2024.